

Learning for (Generalized) Planning

Blai Bonet

Agenda

- What is generalized planning about?
- Learning isn't optional
- Learning policies:
 - ▷ Abstractions
 - ▷ Learning policies with symbolic methods
 - ▷ Learning policies with neural methods
- Learning STRIPS models:
 - ▷ Different assumptions, different algorithms
 - ▷ Symbolic learning: two approaches
 - ▷ Neural learning with transformers

STRIPS/PDDL: Standard Language for Classical Planning

- Grounded **STRIPS**:
 - ▷ Language for planning problems in compact form
 - ▷ States are subsets of ground atoms (facts): those that are true
 - ▷ Actions induce transitions among states:
 - **Prec**: atoms that must be true for action to be applicable
 - **Add**: atoms that become true (added)
 - **Del**: atoms that become false (removed)
- First-order **PDDL** stands for **P**lanning **D**omain **D**escription **L**anguage
- Developed for **I**nternational **P**lanning **C**ompetetion (**IPC**); since 1998
- Problems $P = \langle D, I \rangle$ in PDDL specified in two parts:
 - ▷ **Domain** D contains action **schemas** over **predicates**; possibly types
 - ▷ **Instance** I contains **objects**, **initial situation**, and **goal**
 - ▷ **Grounding** domain D with objects in I yields STRIPS problem

PDDL Example: Gripper Domain and Instance with 2 Balls

```
(define (domain Gripper)
  (:requirements :typing)
  (:types room ball gripper)
  (:constants left right - gripper)
  (:predicates (at-robot ?r - room) (at ?b - ball ?r - room)
               (free ?g - gripper) (carry ?o - ball ?g - gripper))

  (:action move
    :parameters (?from ?to - room)
    :precondition (at-robot ?from)
    :effect (and (at-robot ?to)
                 (not (at-robot ?from))))

  (:action pick
    :parameters (?obj - ball ?room - room ?gripper - gripper)
    :precondition (and (at ?obj ?room) (at-robot ?room) (free ?gripper))
    :effect (and (carry ?obj ?gripper)
                 (not (at ?obj ?room)) (not (free ?gripper))))

  (:action drop
    :parameters (?obj - ball ?room - room ?gripper - gripper)
    :precondition (and (carry ?obj ?gripper) (at-robot ?room))
    :effect (and (at ?obj ?room) (free ?gripper)
                 (not (carry ?obj ?gripper))))

(define (problem Gripper-2-2)
  (:domain Gripper)
  (:objects rA rB - room B1 B2 - ball)
  (:init (at-robot rA) (free left) (free right) (at B1 rA) (at B2 rA))
  (:goal (and (at B1 rB) (at B2 rB))))
```

PDDL Example: Blocks and Instance with 3 Blocks

```
(define (domain Blocks3ops)
  (:predicates (clear ?x) (ontable ?x) (on ?x ?y))

  (:action stack
    :parameters (?x ?y)
    :precondition (and (ontable ?x) (clear ?x) (clear ?y) (not (= ?x ?y)))
    :effect (and (on ?x ?y)
                  (not (ontable ?x)) (not (clear ?y))))

  (:action unstack
    :parameters (?x ?y)
    :precondition (and (on ?x ?y) (clear ?x) (not (= ?x ?y)))
    :effect (and (ontable ?x) (clear ?y)
                  (not (on ?x ?y))))

  (:action move
    :parameters (?x ?y ?z)
    :precondition (and (on ?x ?y) (clear ?x) (clear ?z) (not (= ?x ?z)))
    :effect (and (on ?x ?z) (clear ?y)
                  (not (on ?x ?y)) (not (clear ?z))))

(define (problem Blocks-3-1)
  (:domain Blocks3ops)
  (:objects A B C)
  (:init (clear A) (clear B) (on A C) (ontable C) (ontable B))
  (:goal (and (on C B) (on B A))))
```

Generalized Planning

- Get policy/strategy that solves **multiple instances** of same domain
- Class $\mathcal{Q}$ of **grounded** problems $P = \langle D, I \rangle$ over **common** domain D
- $\mathcal{Q}$ often infinite, not “representable”
- **Challenges:**
 - ▷ No single shared set of actions
 - ▷ Unbounded “branching factor” (bounded at each state, though)
 - ▷ **How to define and specify policies/strategies?**
 - ▷ **Policies must solve all instances in class $\mathcal{Q}$**
- **Example:** get policy that solves every instance of Blocks
- Is a (standard) planner a solution?

Learning Isn't Optional

- $\mathcal{Q}$ often infinite, not “representable” (what does this mean?)
- Not every specification over a domain language corresponds to “real state”
 - ▷ Logistics: $[(\text{in } p1 \text{ airplane2}), (\text{at } p1 \text{ loc5})]$, etc
 - ▷ Blocks: $[(\text{on } A \ B), (\text{on } A \ C)], [(\text{on } A \ B), (\text{on } B \ C), (\text{on } C \ A)]$, etc
- Domain doesn't come with specification of intended states; FOL is not sufficient (e.g. Blocks)
- Known (descriptive complexity):
 - ▷ PTIME equals FOL+LFP (ordered structures)
 - ▷ NP equals $\exists\text{SO}$ (ordered structures)
 - ▷ PSPACE equals FOL+PFP (ordered structures)
- Avoid specification $\rightarrow$ synthesis not available $\rightarrow$ learning needed

Implications

- **Inductive:** find policy for training instances; expect it generalizes on entire $\mathcal{Q}$
- When $\mathcal{Q}$ is infinite (and discrete): extremal “out-of-distribution” generalization
- Sometimes one can show policy is correct, but this is not guaranteed

Learning General Policies

Examples Generalized Policies (Informal Description)

- **Gripper:** move all balls from room A to room B

```
1: if robot in room  $B$  then move robot to room  $A$   
2: while there are balls in room  $A$  do  
3:   while there are balls in room  $A$  and free capacity do  
4:     Pick a ball in room  $A$   
5:     Move robot to room  $B$   
6:     while there are balls being held do  
7:       Drop ball in room  $B$   
8:     Move robot to room  $A$ 
```

- **Blocks3ops:** build towers given in goal description

```
1: % Move all blocks to table  
2: while there is block  $x$  on block  $y$  do  
3:   Move block  $x$  to table  
4: % Build towers from the bottom up  
5: while there is still an incomplete tower do  
6:   Let  $on(x, y)$  be a goal such that  $y$  is well-placed and  $x$  in on the table  
7:   Stack  $x$  on top of  $y$ 
```

Abstractions

- Need different abstractions because:
 - ▷ Number and name of objects differ across instances
 - ▷ No single shared set of (grounded) actions
 - ▷ Unbounded “branching factor” (bounded at each state, though)
- Policy language yields required abstractions:
 - ▷ States abstracted with features
 - ▷ Transition abstracted with rules
 - ▷ State trajectories (aka temporal abstractions; options in RL) abstracted also with rules
- **Result:** policies agnostic to number and names of objects and transitions

High-Level View of Policies and Solutions

- Policy π is **classifier** that separates transitions (edges) into **good or bad**: when at a state, take any **good transition**
- A π -trajectory is $(s_0, s_1, s_2, \dots)$ such that each (s_i, s_{i+1}) is good transition
- π solves P if **each** maximal π -trajectory seeded at initial state ends in goal state;
- Equivalently, π solves P (with initial state s_0) iff
 - ▷ **Safe**: no π -trajectory seeded at s_0 leads to dead end
 - ▷ **Closed**: if $(s_0, \dots, s_n)$ is π -traj. and s_n isn't goal, exists good transition (s_n, s')
 - ▷ **Acyclic**: there is no π -trajectory $(s_0, \dots, s_\ell, \dots, s_k)$ with $s_\ell = s_k$ and $\ell < k$
- π solves $\mathcal{Q}$ if it solves each P in $\mathcal{Q}$
- **Learning**: for finite (and small) $\mathcal{Q}' \subseteq \mathcal{Q}$, find π that solves $\mathcal{Q}'$; expect solves $\mathcal{Q}$

State Abstractions

Feature-based abstractions, defined by small set of Boolean or numerical features:

- Boolean feature f maps states s into Boolean values $f(s)$
- numerical feature n maps states s into real values $n(s)$ (eg. non-negative integers)

Features obtained from **pool of concepts and roles**, constructed inductively from **primitive** concepts and roles in planning domain D using a **grammar**.

- Concept C at state s gets denotation $C[s]$ which is **subset of objects**
- Role R at state s gets denotation $R[s]$ which is **subset of object pairs**
- Concept C defines feature $f_C(s) = |C[s]|$ (which is Boolean if always $\in \{0, 1\}$)

Feature Grammar and Interpretation

- **Atoms:**

- ▷ Native concepts C for unary predicates, and roles R for binary predicates in domain
- ▷ Native concepts/roles at goal: C/g and R/g
- ▷ Universe $\top(s) = U(s)$ consists of all objects
- ▷ $\text{nominal}(\text{const})(s) = \{\text{const}\}$ for **PDDL constant** const

- **Concepts from concepts:**

- ▷ $\text{union}(C, C')(s) = C(s) \cup C'(s)$
- ▷ $\text{inter}(C, C')(s) = C(s) \cap C'(s)$
- ▷ $\text{diff}(C, C')(s) = C(s) \setminus C'(s)$
- ▷ $\text{compl}(C)(s) = U(s) \setminus C(s)$

- **Roles from roles:**

- ▷ $\text{inv}(R)(s) = \{(o, o') \mid (o', o) \in R(s)\}$

- **Concepts from roles:**

- ▷ $\text{exists}(R, C)(s) = \{o \mid \exists o'[(o, o') \in R(s) \wedge o' \in C(s)]\}$
- ▷ $\text{forall}(R, C)(s) = \{o \mid \forall o'[(o, o') \in R(s) \implies o' \in C(s)]\}$
- ▷ $\text{proj}(R, 1)(s) = \{o \mid \exists o'[(o, o') \in R(s)]\}$ (similarly, $\text{proj}(R, 2)$)

- **Roles from concepts:**

- ▷ $\text{restr}(R, 1, C)(s) = \{(o, o') \in R(s) \mid o \in C(s)\}$ (similarly, $\text{restr}(R, 2, C)$)
- ▷ $\text{equals}(R, R')(s) = \{o \mid \forall o'[(o, o') \in R(s) \leftrightarrow (o, o') \in R'(s)]\}$

- Size (resp. depth) complexity of C or R is size (resp. depth) of parse tree

Examples of State Abstractions

- **Gripper:** if goal is to move non-specific balls from one room to another:
 - ▷ Robot at target room: `exists(at/g, at-robot)`
 - ▷ Held balls: `exists(carry, T)`
 - ▷ Balls not in target room: `compl(equals(at/g, at))`
 - ▷ Free grippers (capacity): `free`
- **Blocks3opsTable** (where table is also object):
 - ▷ Clear blocks: `clear`
 - ▷ Blocks not in final position: `compl(equals(on/g, on))`
 - ▷ Blocks st all objects below are well-placed: `forall(tc(on), equals(on/g, on))`

Construction of Feature Pool

- Two concepts C and C' (likewise, roles R and R') are:
 - ▶ Equivalent for state pair (s, s') , written $C \sim_{s,s'} C'$, iff $C[s] = C[s']$
 - ▶ Equivalent for set $\mathcal{P}$ of state pairs, written $C \sim_{\mathcal{P}} C'$, iff $C \sim_{s,s'} C'$ for all $(s, s') \in \mathcal{P}$
- (Naive) algorithm for constructing pool of features of given complexity K :

```
1: Let  $\mathcal{I} := \emptyset$ 
2: while true do
3:   Let  $\mathcal{I}' := \emptyset$ 
4:   for  $S \subseteq \mathcal{I}$  with  $|S| \leq 2$  of total complexity  $< K$  do
5:     Create concept/role  $e$  from  $S$  (e.g. forall concept or inverse role)
6:     if  $e$  isn't equivalent to any item in  $\mathcal{I} \cup \mathcal{I}'$  then
7:        $\mathcal{I}' := \mathcal{I}' \cup \{e\}$ 
8:   if  $\mathcal{I}' = \emptyset$  then break
9:    $\mathcal{I} := \mathcal{I} \cup \mathcal{I}'$ 
```
- Concept C (role R) defines feature $|C|$ ($|R|$) which is Boolean if always in $\{0, 1\}$

Extended Expressivity

- Numerical distance features:

- ▷ $\text{dist}(C, R, C')(s) \leq n$ iff $\exists(o_1, o_2, \dots, o_n)$ st $o_1 \in C$, $o_n \in C'$, and $(o_i, o_{i+1}) \in R$
- ▷ $\text{dist}(C, R, C')(s) = n$ iff $\text{dist}(C, R, C')(s) \leq n$ and $\text{dist}(C, R, C')(s) \not\leq n-1$

- Non-C2 features:

- ▷ Transitive closure: $\text{tc}(R)(s) = \{(o, o') \mid R^+(o, o')\}$
- ▷ Composition: $\text{comp}(R, R')(s) = \{(o, o') \mid \exists o''[(o, o'') \in R(s) \wedge (o'', o') \in R']\}$
- ▷ ...

Rules: Syntax and Semantics

Rules have form $C \mapsto E$ where C and E are sets of conditions and effects:

- **Conditions:** $\{p, \neg p\}$ for Booleans p ; $\{n > 0, n = 0\}$ for numerals n
- **Effects:** $\{p, \neg p, p?\}$ for Booleans p ; $\{n\uparrow, n\downarrow, n?\}$ for numerals n

Example: $\{H, n > 0\} \mapsto \{\neg H, n\downarrow\}$ is rule over Boolean H and numeral n

Transition (s, s') is **compatible** with rule $C \mapsto E$ iff:

- Conditions:
 - ▷ if p (resp. $\neg p$) in C , the $p(s) = 1$ (resp. $p(s) = 0$)
 - ▷ if $n = 0$ (resp. $n > 0$) in C , the $n(s) = 0$ (resp. $n(s) > 0$)
- Effects:
 - ▷ if p (resp. $\neg p$) in E , then $p(s') = 1$ (resp. $p(s') = 0$)
 - ▷ if $n\uparrow$ (resp. $n\downarrow$) in E , then $n(s') > n(s)$ (resp. $n(s') < n(s)$)
 - ▷ if p (resp. n) **not in** E , then $p(s') = p(s)$ (resp. $n(s') = n(s)$)

Rule-Based Policies

- Policy π is set of rules over the feature Φ mentioned in rules
- Transition (s, s') compatible with π (“good”) if compatible with some rule in π
- Trajectory $(s_0, s_1, \dots)$ is π -trajectory if each (s_i, s_{i+1}) is in π

Example: Policy for navigation problem where rewards in grid must be picked

$$\{r > 0, d = 0\} \mapsto \{r\downarrow, d\uparrow\}$$

$$\{r > 0, d > 0\} \mapsto \{d\downarrow\}$$

where $r \equiv |\text{at}|$ is number of rewards in grid; $d = \text{dist}(\text{at-robot}, \text{adj}, \text{exists}(\text{at}, \top))$ is distance to closest reward

Policy is safe, closed and acyclic: it solves any instance!

Principles for Learning Rule-Based Policies

- **Projection** of transition (s, s') on features in Φ results in rule $C \mapsto E$ where
 - ▷ C is Boolean valuation of features in Φ on state s
 - ▷ E is qualitative change of features in Φ across (s, s')

Fact. Transition (s, s') is compatible with its projection $C \mapsto E$

- Transitions (s, s') and (t, t') **separated** by feature f that changes differently across (s, s') and (t, t')

Example: Consider features $\Phi = \{n, at_{target}\}$ in Gripper:

- $n = |\text{exists}(\text{carry}, \top)|$ (# balls carried)
- $at_{target} = |\text{exists}(\text{at}, \text{proj}(\text{at/g}, 2))|$ (# balls at target room)
- Projection of (drop B1 rB left) on Φ is $\{n > 0, at_{target} = 0\} \mapsto \{n \downarrow, at_{target} \uparrow\}$
- n and at_{target} separate (drop B1 rB left) and (move rB rA)

SAT-Based Learning [Francès et al., 2021]

- **Input:** transitions $\mathcal{T}$ over states $\mathcal{S}$, feature pool $\mathcal{F}$, distance budget K
- **Output:** Propositional theory $\text{Th}(\mathcal{T}, \mathcal{F}, K)$ over $\{\text{Select}(f), \text{Good}(s, s'), \text{Dist}(s, k)\}$

- A1. One of $\{\text{Dist}(s, k) : k = 0, 1, \dots, K\}$ [for non-dead end $s \in \mathcal{S}$]
A2. $\bigvee_{(s, s') \in \mathcal{T}} \text{Good}(s, s')$ [for non-dead end $s \in \mathcal{S}$]
A3. $\neg \text{Good}(s, s')$ [for dead end $s' \in \mathcal{S}, (s, s') \in \mathcal{T}$]
A4. $\text{Good}(s, s') \wedge \text{Dist}(s, \ell) \wedge \text{Dist}(s', k) \implies k < \ell$ [for $(s, s') \in \mathcal{T}$]
A5. $\text{Good}(s, s') \wedge \neg \text{Good}(t, t') \implies \bigvee_{f: \Delta_f(s, s') \neq \Delta_f(t, t')} \text{Select}(f) [(s, s'), (t, t') \in \mathcal{T}]$
where $\Delta_f(s, s') \in \{=, +, -\}$ is the change of f across (s, s')

Let $\Pi(\mathcal{T}, \mathcal{F}, K)$ be set of policies over $\mathcal{F}$ that solve non-dead-end states in $\mathcal{S}$ in at most K steps. $\mathcal{T}$ is **closed** iff for each non dead-end $s \in \mathcal{S}$, there is $(s, s') \in \mathcal{T}$ where s' is not dead end

Theorem. Let $\mathcal{T}$ be closed. $\Pi(\mathcal{T}, \mathcal{F}, K)$ is non-empty iff $\text{Th}(\mathcal{T}, \mathcal{F}, K)$ is SAT. If ν satisfies $\text{Th}(\mathcal{T}, \mathcal{F}, K)$, the projection of Good_ν on $\mathcal{F}_\nu$ is such policy where $\text{Good}_\nu = \{(s, s') \in \mathcal{T} : \nu \models \text{Good}(s, s')\}$ and $\mathcal{F}_\nu = \{f \in \mathcal{F} : \nu \models \text{Select}(f)\}$

Learning Algorithm

- Given small set $\{P_1, \dots, P_k\}$ of small training instances (e.g. $k = 1$ or $k = 2$)
- Get pool $\mathcal{F}$ of feature of complexity bounded by $K_{\mathcal{F}}$ (parameter)
- Obtain reachable transitions $\mathcal{T}$ and states $\mathcal{S}$; construct theory $\text{Th}(\mathcal{T}, \mathcal{S}, K)$
- Call Max-SAT solver over $\text{Th}(\mathcal{T}, \mathcal{S}, K)$ to prefer small over large policies
- Project transitions in $\mathcal{T}$ over solution ν to get policy, and evaluate
- Optimizations:
 - ▷ Constraint $\text{Dist}(s, k)$ for $V^*(s) \leq k \leq \delta V^*(s)$, where $\delta = 2$
 - ▷ Pool $\mathcal{F}$ defines equivalence relation among states; can formulate theory using representatives
 - ▷ Number of separation clauses (A5) grows quadratically in $\mathcal{T}$: generate them incrementally

Results

- Open-WBO Weighted Max-SAT solver [Martins, Manquinho, and Lynce 2014]
- Pool complexity $K_{\mathcal{F}} = 8$

Domain	$ Q $	dim	$\mathcal{T}$	$\mathcal{T}/\sim$	$d_{\max}$	$ \mathcal{F} $	vars	clauses	time	SAT	compl.	$ \Phi $	K^*	$ \pi $
Blocks4ops-clear	1	5	1,161	55	7	532	7.9K	247.7K	6	< 1	8	3	4	3
Blocks4ops-on	1	5	1,852	329	10	1,412	17.3K	376.6K	33	22	13	5	5	7
Gripper	1	4	1,140	61	12	835	6.5K	102.6K	2	< 1	9	3	4	4
Reward	1	5×5	432	361	15	514	5.5K	214.9K	2	< 1	7	2	6	2
Delivery	2	4×4	42,473	5,442	56	1,373	753.4K	38.2M	3,071	2,902	30	4	14	6
Visitall	1	3×3	2,396	310	8	188	13.9K	244.5K	3	< 1	7	2	5	1
Spanner	3	(6, 10)	10,777	96	19	764	85.0K	2.2M	32	< 1	9	3	6	2
Miconic	2	(4, 7)	4,706	4,636	14	1,073	23.8K	23.6M	41	61	11	4	5	5
Blocks3ops	2	5	4,275	4,275	8	1,896	22.1K	9.3M	80	40	11	3	6	1

#training instances, dimension, #transitions, #equiv. classes modulo features, pool size, size SAT theory in vars and clauses, total time, time in SAT, complexity and size of selected features, effective pool complexity, and number of rules

Example Learned Policy: Blocks3opsTable

- Domain with 3 operators, predicates `clear/1` and `on/2`, constant `table`
- Features:
 - ▷ $c = |\text{clear}|$ [#objs clear]
 - ▷ $t' = |\text{compl}(\text{equals}(\text{on/g}, \text{on}))|$ [#objs **not** in target]
 - ▷ $bwp = |\text{forall}(\text{tc}(\text{on}), \text{equals}(\text{on/g}, \text{on}))|$ [#objs st all below are well-placed]
- Learned policy:
$$\{c > 0, t' > 0, bwp > 0\} \mapsto \{c \uparrow\}$$
$$\{c > 0, t' > 0, bwp > 0\} \mapsto \{c \uparrow, bwp \uparrow, t' ?\}$$
$$\{c > 0, t' > 0, bwp > 0\} \mapsto \{c \uparrow, t' \downarrow\}$$
$$\{c > 0, t' > 0, bwp > 0\} \mapsto \{c \downarrow, t' \downarrow\}$$
- Policy is safe, closed, and acyclic [Francès et al., 2021]

Structural Termination

Consider the policy:

- $\{H\} \mapsto \{\neg H\}$ (H flips value from **true** to **false**)
- $\{\neg H, n > 0\} \mapsto \{H, n \downarrow\}$ (H flips value from **false** to **true**, and n **decrements**)

The policy is **acyclic** by design:

- ▷ each rule flips the value of the Boolean H
- ▷ each flip from $\neg H$ to H comes with a decrement of n which no rule increments

For non-negative integer-valued features (as those induced by concepts/roles), there is a **finite** number of flips

Policy π is **structurally terminating** if it cannot generate a cycle independently of the denotation of its features. Termination is implied by how features change

Monotone Features

Let R be a set of rules over the features in Φ :

- Numerical n is **monotone in** R if no rule increases it, or no rule decreases it
- Boolean p is **monotone in** R if no rule makes it true, or no rule makes it false

For numerical (or Boolean) g , consider the following subsets of rules:

- $\varrho(R, g, =) = \{r \in R : \{g\uparrow, g\downarrow\} \cap \text{eff}(r) = \emptyset\}$
- $\varrho(R, g, 0) = \{r \in \varrho(R, g, =) \mid 'g > 0' \notin \text{cond}(r)\}$
- $\varrho(R, g, 1) = \{r \in \varrho(R, g, =) \mid 'g = 0' \notin \text{cond}(r)\}$

Feature f is **monotone given** g iff it is monotone in $\varrho(R, g, 0)$ and $\varrho(R, g, 1)$

Stratified Policies

Policy π is **1-stratified** iff

- Every rule entails the change of some feature
- There is ranking κ of the features such that
 - ▷ if $\kappa(f) = 0$, feature f is monotone in π
 - ▷ if $\kappa(f) > 0$, there is g such that $\kappa(g) < \kappa(f)$ and f is monotone given g

Theorem. If π is 1-stratified, it's acyclic. Testing 1-stratification takes polytime

Suppose not. Consider **infinite** $s_0, s_1, s_2, \dots$. Since integer and bounded (in instance) features, some f increases **and** decreases i.o. Choosing such f of **min rank** yields contradiction ($\Rightarrow \Leftarrow$)

Testing 1-stratification. Construct κ . Choose f that is monotone in π . Set $\kappa(f) = 0$.

At stage i , choose non-ranked f that is monotone given g already chosen one. Set $\kappa(f) = i$. There are $\mathcal{O}(n^2)$ possibilities for f and g . Testing if such a pair works takes time linear in $|\pi|$.

Example of Stratified Policy

- Policy over $\{n, m, A\}$ where n and m are numerical, and A is Boolean:

$$\{n > 0\} \mapsto \{n\downarrow, m?\}$$

$$\{m > 0\} \mapsto \{m\downarrow\}$$

$$\{A, m > 0\} \mapsto \{\neg A\}$$

$$\{\neg A, m = 0\} \mapsto \{A\}$$

- Stratification $\kappa(n) = 0$, $\kappa(m) = 1$, and $\kappa(A) = 2$:
 - ▷ n is monotone: no rule increases n
 - ▷ m is monotone given n : removing the first rule, no rule increases m
 - ▷ A is monotone given m : A is monotone in $\varrho(\pi, m, 0)$ and in $\varrho(\pi, m, 1)$
- Suppose infinite π -trajectory $s_0, s_1, s_2, \dots$
 - ▷ After finite prefix, the sequence only executes rules r2–r4
 - ▷ even further, the sequence $s_k, s_{k+1}, s_{k+2} \dots$ only executes rules in r3–r4
 - ▷ yet value of m is constant in such prefix ... impossible!

k -Stratified Policies

Similar notion but where conditional monotonicity defined in terms of k features

Theorem. If π is k -stratified, it's acyclic. Testing k -stratification is exponential in k , but polynomial in $|\pi|$ and $|\Phi|$

Learning Terminating Policies From Examples

- SAT-based learning solves two (coupled) combinatorial problems:
 - ▷ Choosing good transitions for alive states
 - ▷ Choosing features that separate good from bad transitions
- Examples provide **good transitions**; they come from tutor (planner, human, ...)
- Policy rules obtained by **projecting** good transitions into selected features
- **Task:** select features $\{f_1, f_2, \dots, f_k\}$ that make up a “terminating set”
- **No SAT/ASP; want something fast and scalable; switch to hitting sets!**

Hitting Set Problem

- Classical NP-Complete problem (one of Karp's problems):
 - ▷ **Input:** collection $S_1, S_2, \dots, S_n$ of subsets over universe $U = \cup_{1 \leq i \leq n} S_i$
 - ▷ **Input:** non-negative integer $k \geq 0$
 - ▷ **Question:** is there $H \subseteq U$ such that $|H| \leq k$, and $S_i \cap H \neq \emptyset$ for $1 \leq i \leq n$?
- Dual of Set Cover, non-approximable, FPT for different parameters
- Weights $w : U \rightarrow \mathbb{N}$, and ask for H of weight $\sum_{e \in H} w(e) \leq k$
- **Approximation greedy algorithm** chooses element that hits maximum number of not-yet-hit subsets normalized by weight:

```
1:  $H := \emptyset$ 
2: while there are subsets not hit by  $H$  do
3:   Choose  $e$  with  $\max \text{score}(e) \doteq |\{i \mid e \in S_i, S_i \cap H = \emptyset\}| / w(e)$ 
4:    $H := H \cup \{e\}$ 
```

[Approximation ratio of $\ln n + o(1)$ for n subsets]

Base for Learning Terminating Policies using Hitting Sets

- **Inputs:** Examples as set $\mathcal{T}$ of transitions, and (possibly) large pool $\mathcal{F}$ of features
- **Output:** Subset $\Phi \subseteq \mathcal{F}$ of features on which to project the transitions in $\mathcal{T}$
- Set Φ of features must satisfy set of **requirements**
- Requirements are expressed **extensionally** as subsets of features:
 - ▷ For each edge $e \in \mathcal{T}$, some feature f must change: $R_e = \{f \in \mathcal{F} \mid f \text{ changes across } e\}$
 - ▷ (whatever suits you . . . more about this later)

Base for Learning Terminating Policies using Hitting Sets

- **Inputs:** Examples as set $\mathcal{T}$ of transitions, and (possibly) large pool $\mathcal{F}$ of features
- **Output:** Subset $\Phi \subseteq \mathcal{F}$ of features on which to project the transitions in $\mathcal{T}$
- Set Φ of features must satisfy set of **requirements**
- Requirements are expressed **extensionally** as subsets of features:
 - ▷ For each edge $e \in \mathcal{T}$, some feature f must change: $R_e = \{f \in \mathcal{F} \mid f \text{ changes across } e\}$
 - ▷ (whatever suits you . . . more about this later)
- Hitting set formulation:

Input: $\mathcal{T}$, pool $\mathcal{F}$ (each with comp. $w(f)$), and set $\mathcal{R}$ of requirements (each is $\mathcal{F}$ -subset)

- 1: $\Phi := \emptyset$
- 2: **while** there is some requirement not hit by Φ **do**
- 3: Let $\mathcal{F}_{\text{enabled}} \subseteq \mathcal{F}$ be the set of “features enabled by Φ ”
- 4: Choose f in $\mathcal{F}_{\text{enabled}}$ with $\max \text{score}(f) \doteq |\{R \in \mathcal{R} \mid f \in R, R \cap \Phi = \emptyset\}| / w(f)$
- 5: $\Phi := \Phi \cup \{f\}$

where $\mathcal{F}_{\text{enabled}} = \{f \in \mathcal{F} \mid f \text{ is monotone, or monotone given some } g \text{ in } \Phi\}$

- Resulting policy π is **projection** of $\mathcal{T}$ over Φ ; $\pi = \text{proj}(\mathcal{T}, \Phi)$

Time to Evaluate the Policy

- What can go wrong?

- Policy generates a cycle

Impossible. By design, the policy is terminating!

Time to Evaluate the Policy

- What can go wrong?
- **Policy generates a cycle** **Impossible. By design, the policy is terminating!**
- Policy reaches state s where no transition (s, t) is compatible with a rule:
 - ▷ It can happen. **Policy isn't necessarily closed**
 - ▷ Ask the tutor (planner) to generate a new transition $e = (s, t)$
 - ▷ Augment transitions and requirements: $\mathcal{T} := \mathcal{T} \cup \{e\}$ and $\mathcal{R} := \mathcal{R} \cup \{R_e\}$

Time to Evaluate the Policy

- What can go wrong?
- Policy generates a cycle **Impossible. By design, the policy is terminating!**
- Policy reaches state s where no transition (s, t) is compatible with a rule:
 - ▷ It can happen. **Policy isn't necessarily closed**
 - ▷ Ask the tutor (planner) to generate a new transition $e = (s, t)$
 - ▷ Augment transitions and requirements: $\mathcal{T} := \mathcal{T} \cup \{e\}$ and $\mathcal{R} := \mathcal{R} \cup \{R_e\}$
- Policy generates trajectory $(s_0, s_1, \dots, s_n)$ where s_n is dead end state:
 - ▷ It can also happen. **Policy isn't necessarily safe**
 - ▷ **Find guilty transition** $b = (s_i, s_{i+1})$ for the purpose of marking it as a **bad transition**
 - ▷ For each transition $e \in \mathcal{T}$, **add requirement** $R_{e,b} = \{f \in \mathcal{F} \mid \Delta_f(e) \neq \Delta_f(b)\}$
 - ▷ Number of requirements grows by $|\mathcal{T}|$
 - ▷ **Guilty transition:** (s_i, s_{i+1}) is “first transition” such that all previous belong to $\mathcal{T}$

Results (Successes)

Domain	$ Q $	$ F $	Outer	Inner	Last outer iteration							Time in seconds			
					$ Q' $	Inner*	$ T $	$ B $	$\ H\ $	$ H $	$ \pi $	Prep.	Alg.	Verif.	Total
3puzzle	19	1,000	1	1	1	1	6	0	12	3	5	0.0	0.0	0.1	2.4
Blocks4ops-clear	53	13,579	1	1	1	1	7	0	14	2	2	0.6	0.0	0.1	10.4
Delivery-1pkg	169	5,765	1	1	1	1	9	0	18	4	4	0.2	0.0	3.2	24.6
Gripper	5	6,154	1	1	1	1	19	0	38	3	4	0.4	0.2	3.9	8.2
Reward	207	249,122	1	1	1	1	17	0	34	2	2	26.1	33.7	6.9	152.5
Visitall	460	146,085	1	1	1	1	20	0	40	2	3	15.6	1.4	991.5	1,110.1
Childsnack	10	2,900	1	2	1	2	15	0	30	6	8	0.5	0.0	32.1	35.6
Spanner-1nut	90	8,001	1	2	1	2	6	1	19	3	3	0.5	0.0	0.5	11.7
Logistics-1truck	67	262,509	1	17	1	17	32	0	64	5	8	905.5	803.5	2.7	1,926.6
Barman-1cocktail-1shot	90	69,040	1	21	1	21	32	2	133	11	22	243.1	539.9	216.1	1,049.9
Blocks4ops-on	94	183,322	2	2	1	1	10	0	20	4	7	24.2	1.9	1.1	108.2
Spanner	270	13,679	2	3	1	2	10	1	31	3	3	1.9	0.3	176.0	210.4
Delivery	397	13,904	4	21	1	3	23	0	46	4	5	42.1	6.6	23.0	135.3
Ferry	180	8,547	5	13	1	6	17	0	34	4	5	8.6	3.9	3.6	39.3
Miconic	360	107,785	9	30	1	6	20	0	40	4	5	253.8	73.7	50.8	502.8
8puzzle-1tile-fixed	18	11,230	7	38	4	3	40	0	80	8	14	44.7	25.5	1.8	81.5
8puzzle-1tile	16	12,417	8	52	3	6	40	0	80	8	18	69.1	35.6	2.2	117.8
Blocks4ops	5	100,897	10	66	3	5	81	0	162	7	24	5,349.4	7,159.4	19.6	12,863.9
Sokoban-1stone-7x7	8	115,214	12	309	5	15	94	4	671	15	50	14,248.4	12,712.3	896.1	28,196.5
Logistics-1pkg	24	225,518	15	138	4	8	56	0	112	7	16	6,936.5	5,913.8	416.7	16,136.6
Zenotravel-1plane	73	24,959	28	266	7	3	122	0	244	7	18	2,039.2	277.3	2,905.9	5,406.4

#instances, #features, #outer/inner iterations, #instances seen, #good/bad transitions, size of hs. problem, size of hs, policy size, and preprocessing, algorithm, and verification times

Results (Failures)

Domain	$ Q $	$ F $	Outer	Inner	Last outer iteration					Reason	Time in seconds			
					$ Q' $	Inner*	$ T $	$ B $	$\ H\ $		Prep.	Alg.	Verif.	Total
Rovers	608	190,064	1	1	1	1	25	0	50	Edge	17.2	0.5	0.0	225.7
Tidybot-opt11-strips	8	59,402	1	1	1	1	40	0	80	Edge	7.7	0.2	0.0	119.0
Tpp	11	14,128	1	1	1	1	201	0	402	Edge	9.3	0.2	0.0	237.1
8puzzle-2tiles	16	4,458	1	2	1	2	20	0	40	Edge	0.8	0.1	0.1	6.3
Hiking	180	16,723	1	2	1	2	8	0	16	Edge	1.4	0.0	4.0	190.0
Depot	18	255,079	1	17	1	17	119	0	238	Edge	4,771.5	379.5	415.2	15,511.7
Freecell	65	146,428	1	35	1	35	133	19	2,964	Timeout	—	—	—	—
Barman-1cocktail	270	69,040	1	94	1	94	105	6	868	Edge	3,056.5	5,474.4	36.4	8,769.3
Tetris-opt14-strips	16	37,496	1	142	1	142	180	1	550	Edge	5,109.8	3,458.8	7,157.7	16,500.1
Satellite	950	171,475	2	209	1	157	168	0	334	Timeout	—	—	—	—
Driverlog	381	172,818	6	152	1	16	31	0	62	Edge	5,768.1	1,822.2	52.3	8,088.9
Zenotravel-1person	80	42,271	12	385	1	77	80	3	507	Edge	2,262.9	350.4	1,359.2	4,701.4
Logistics	282	51,418	11	72	4	5	99	0	198	Edge	987.5	169.3	15.4	1,310.9
Blocks3ops	392	236,954	17	102	7	8	106	0	206	Timeout	—	—	—	—

#instances, #features, #outer/inner iterations, #instances seen, #good/bad transitions, size of hs. problem, **failure reason**, and preprocessing, algorithm, and verification times

Recursive Termination

- Conditional monotonicity is one way to get structural termination
- **Recursive termination is more powerful:**

```
1: function SOLVE( $\mathcal{T}$ )
2:   if  $\mathcal{T}$  is empty then Success
3:   Choose  $f$  that is monotone in  $\mathcal{T}$  and changes some edge           [Let  $\mathcal{T}_f = \mathcal{T}$ ]
4:   if no such  $f$  then Failure
5:   Let  $\mathcal{T}_f^*$  be edges in  $\mathcal{T}$  where  $f$  doesn't change                 [Notice  $\mathcal{T}_f^* \subsetneq \mathcal{T}_f$ ]
6:   Partition  $\mathcal{T}_f^* = \mathcal{T}_f^*[0] \cup \mathcal{T}_f^*[1]$  where  $\mathcal{T}_f^*[\nu] = \{(s, t) \in \mathcal{T}_f^* \mid s \models f = \nu\}$ 
7:   Recurse into  $\mathcal{T}_f^*[0]$  and  $\mathcal{T}_f^*[1]$ 
```

- **Theorem.** Recursive termination implies structural termination
- Easy change **enumerates all solutions**; other variations are possible

Learning Policies with Neural Methods

Graph Neural Networks (GNNs)

- Neural architecture capable of processing graphs $G = (V, E)$ of **any size**
- Primary function is to compute **node (vertex) embeddings**, each a vector in $\mathbb{R}^d$
- Architecture characterized by width (embedding dimension), depth (num. layers), (permutation invariant) aggregation and combination functions:

$$\begin{aligned} \mathbf{f}_0(u) &= \mathbf{e}_t \\ \mathbf{f}_{k+1}(u) &= \text{comb}_k(\mathbf{f}_k(u), \text{aggr}_k(\{\{\mathbf{f}_k(v) \mid (u, v) \in E\}\})) \end{aligned}$$

where $\mathbf{e}_t$ is initial embedding for vertices of “atomic type” t

- At the end, final embeddings aggregated and passed through a “readout function”:

$$\mathbf{f}(G) = \text{readout}(\{\{\mathbf{f}_L(u) \mid u \in V\}\})$$

- Functions have **learnable parameters** (sometimes even the initial embeddings)

Concrete GNN architectures

- Important that aggregation is **permutation invariant**; eg. sum, mean, max, etc
- Frequent choice:
 - ▷ Sum/max aggregation of embeddings
 - ▷ Linear combination; non-linear activation (eg ReLU); residual or non-residual
 - ▷ It can be homogeneous (all layers equal) or not; readout is MLP
- For layers $k = 0, 1, \dots, L - 1$:

$$\mathbf{f}_0(u) = \mathbf{e}_t$$

$$\mathbf{f}_{k+1}(u) = \text{ReLU}\left(\mathbf{A}_k \mathbf{f}_k(u) + \mathbf{B}_k \sum_{(u,v) \in E} \mathbf{f}_k(v) + \mathbf{C}_k\right)$$

$$\text{readout}(S) = \text{MLP}\left(\sum_{\mathbf{e} \in S} \mathbf{e}\right)$$

where $\mathbf{A}_k$ and $\mathbf{B}_k$ are $\mathbb{R}^{d \times d}$ matrices, and $\mathbf{e}_t$ and $\mathbf{C}_k$ are $\mathbb{R}^d$ vectors

Implementation and Training of GNNs

- Easily implemented in PyTorch and trained with stoch. gradient descend (SGD)
- As always, data is split into training, validation, and test set:
 - ▷ **Training:** directly used to do actual training (loss and gradient calculation)
 - ▷ **Validation:** after X epochs (epoch is when all data have been seen), net evaluated on validation set (performance and parameters stored)
 - ▷ **Select** model with best performance on validation set
 - ▷ **Evaluate** selected model on test set (withheld from entire training process)
- Training can be supervised, self-supervised, or RL:
 - ▷ **Supervised:** training data consists of pairs (G, L) for graph G and label L
 - ▷ **Self-supervised:** focus on extracting patterns or (latent) representations without needing labels (VAEs, Contrastive, MAEs)
 - ▷ **Reinforcement Learning:** different learning paradigm, GNN acts like “policy” that interacts with environment that yields a **reward signal**
- **Hyperparameter tuning crucial and difficult:** batch size, learning rate, ...

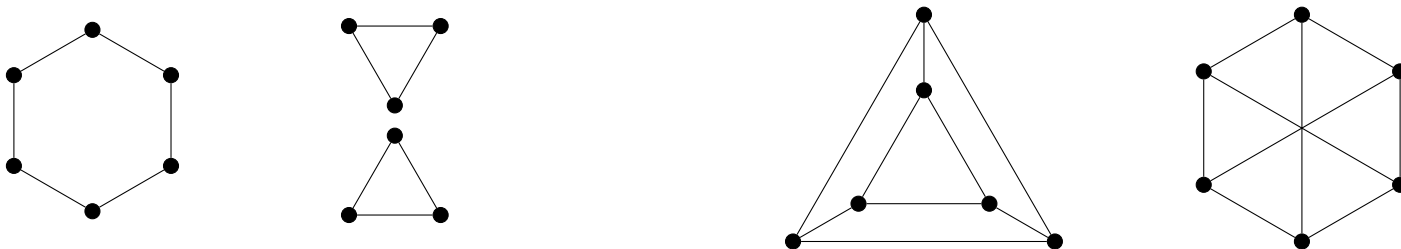
1-WL, Counting Logics, and GNNs

- Clean connection between graph coloring, logic and GNNs
- Weisfeiler-Lehman coloring algorithm (1-WL):

$$f_0(u) = \text{“initial color depending on vertex type”}$$
$$f_{k+1}(u) = \text{RELABEL}(f_k(u), \{\{f_k(v) \mid (u, v) \in E\}\})$$

Iterate until no further refinement possible (max $|V|$ iterations)

- Final coloring f_∞ : $u \sim v$ iff $f_\infty(u) = f_\infty(v)$. Vertex partition is $\{[u] \mid u \in V\}$
- If $G \simeq G'$, they generate equivalent (vertex) partitions; converse is false



Counting Logics and 1-WL

- FO^2 is FOL with 2 variables (restrictive fragment)
- Increase expressivity by allowing **counting quantifiers** of form $\exists^{\geq n} y \phi(x, y)$ meaning there are at least n **distinct objects** y such that $\phi(x, y)$ holds
- Resulting logic is called C^2 (in some variants, the integer n can be quantified and appear in “arithmetic” operations)
- C^2 in graphs:
 - ▷ $\phi(x) = \exists^{\geq 5} y (E(x, y) \wedge \text{red}(y))$ (x has at least 5 red neighbors)
 - ▷ $\psi_0(x) = \text{red}(x)$
 - ▷ $\psi_{d+1}(x) = \psi_d(x) \vee \exists y (E(x, y) \wedge \psi_d(y))$ (x is at dist. $\leq d + 1$ from red)
 - ▷ $\varphi(x) = \exists n, m ((n < m) \wedge \exists^= n y (E(x, y) \wedge \text{red}(y)) \wedge \exists^= m y (E(x, y) \wedge \text{blue}(y)))$
- **Theorem.** For any two graphs G and G' , and two vertices u and v in G :
 - ▷ $G \simeq_{1\text{-WL}} G'$ iff there is no C^2 formula that distinguish G and G'
 - ▷ $f_k(u) = f_k(v)$ iff for every C^2 formula $\phi(x)$ of **rank** k , $G \models \phi(u)$ iff $G \models \phi(v)$

GNNs and 1-WL

- GNNs cannot distinguish graphs/vertices beyond 1-WL:
 - ▷ If $G \simeq_{1\text{-WL}} G'$, then $\mathbf{f}(G) = \mathbf{f}(G')$ for every GNN
 - ▷ If $f_k(u) = f_k(v)$, then $\mathbf{f}_k(u) = \mathbf{f}_k(v)$ for every GNN (even with readouts)
- **Guarded fragment** of C^2 , GC^2 , corresponds to formulas

$$\phi \leftarrow U(x) \mid E(x, y) \mid \neg\phi \mid \phi \vee \phi \mid \exists^{\geq n} y (E(x, y) \wedge \phi(y))$$

- If vertices u and v cannot be distinguished by formulas in GC^2 of **rank** k , then $\mathbf{f}_k(u) = \mathbf{f}_k(v)$ for every GNN

Relational GNN for Planning

- GNNs are designed for graphs; planning states are not graphs
- STRIPS domain D defines **signature** σ with constants and predicates in D
- Planning state s is **relational structure**: denotations given by grounded atoms
- R-GNN[D] for planning states [Stahlberg et al., 2022–today]:

Input: set S of ground atoms, objects O

Output: embeddings $\mathbf{f}_L(o)$ for each object $o \in O$

- 1: Initialize $\mathbf{f}_0(o) = 0^d$ for each object $o \in O$
- 2: **for** layer index $i = 1, 2, \dots, L - 1$ **do**
- 3: **for** each atom $q = R(o_1, o_2, \dots, o_k)$ in S **do**
- 4: $\mathbf{m}_q = \text{MLP}_R(\mathbf{f}_i(o_1), \mathbf{f}_i(o_2), \dots, \mathbf{f}_i(o_k))$
- 5: Send message $\mathbf{m}_{q,o_j} := [\mathbf{m}_q]_j$ to object o_j
- 6: **for** each object o in O **do**
- 7: $\mathbf{f}_{i+1}(o) := \mathbf{f}_i(o) + \text{MLP}_U(\mathbf{f}_i(o), \text{aggr}(\{\{\mathbf{m}_{q,o} \mid q \in S, o \in q\}\}))$

- MLP_R per predicate R ; MLP_U for combining; residual connections
- Aggregation function either sum, smooth-max, or combination

Training and Loss Functions

- Stahlberg et al. [2022a,b]
- Readout generates scalar value (MLP over aggregation of final embeddings)
- Training minimizes **loss over training set** by finding **best** parameter θ
- Loss functions:

$$Loss = \sum_{s \text{ in trainset}} |V^*(s) - V_{\theta}(s)|$$

$$Loss' = \sum_{s \text{ in trainset}} \max \{ 0, [1 + \min_a V_{\theta}(s_a)] - V_{\theta}(s) \}$$

- If $Loss = 0$, $V_{\theta} = V^*$ yields **optimal policies** on training set [Ståhlberg et al., 2022]
- If $Loss' = 0$, $V_{\theta}(s) \geq 1 + \min_a V_{\theta}(s_a)$ yields **policies that solve** training set
- Aim at **generalization performance** over new, test instances

Experiments: Setup

- Experiments aimed at testing generalization (coverage and quality)
- Standard instances from International Planning Competition (IPC)
- Width and depth set at $d = 64$ and $L = 30$: L affects how far messages propagate, d affects number of features in V_θ
- Optimizer Adam with learning rate 0.0002
- **Hardware:** NVIDIA A100 GPUs for up to 12 hours
- During training, loss measured on “validation set”; best θ selected
- Quality measured with respect to **optimal plans**

Experimental Results: Minimizing $Loss'$

- Instance sizes in training, validation and testing by number of objects

Domain	Train	Validation	Test
Blocks	[4, 7]	[8, 8]	[9, 17]
Delivery	[12, 20]	[28, 28]	[29, 85]
Gripper	[8, 12]	[14, 14]	[16, 46]
Logistics	[5, 18]	[13, 16]	[15, 37]
Miconic	[3, 18]	[18, 18]	[21, 90]
Reward	[9, 100]	[100, 100]	[225, 625]
Spanner*	[6, 33]	[27, 30]	[22, 320]
Visitall	[4, 16]	[16, 16]	[25, 121]

- Performance of two deterministic greedy policies: π_{V_θ} **with and without cycle avoidance**

Domain (#)	Deterministic policy π_V with cycle avoidance			Deterministic policy π_V alone		
	Coverage (%)	L	PQ = PL / OL (#)	Coverage (%)	L	PQ = PL / OL (#)
Blocks (20)	20 (100%)	790	1.0427 = 440 / 422 (13)	20 (100%)	790	1.0427 = 440 / 422 (13)
Delivery (15)	15 (100%)	400	1.0000 = 400 / 400 (15)	15 (100%)	404	1.0100 = 404 / 400 (15)
Gripper (16)	16 (100%)	1,286	1.0000 = 176 / 176 (4)	16 (100%)	1,286	1.0000 = 176 / 176 (4)
Logistics (28)	17 (60%)	4,635	9.7215 = 3,665 / 377 (15)	0 (0%)	0	—
Miconic (120)	120 (100%)	7,331	1.0052 = 1,170 / 1,164 (35)	120 (100%)	7,331	1.0052 = 1,170 / 1,164 (35)
Reward (15)	11 (73%)	1,243	1.2306 = 1,062 / 863 (10)	3 (20%)	237	1.1232 = 237 / 211 (3)
Spanner*-30 (41)	30 (73%)	1,545	1.0000 = 1,545 / 1,545 (30)	24 (58%)	940	1.0000 = 940 / 940 (24)
Visitall (14)	14 (100%)	904	1.0183 = 556 / 546 (10)	11 (78%)	631	1.0107 = 471 / 466 (9)
Total (269)	243 (90%)	18,134	1.6410 = 9,014 / 5,493 (132)	209 (77%)	11,619	1.0156 = 3,838 / 3,779 (103)

Understanding and Overcoming Limitations

- Coverage:
 - ▷ **5 out 8 fully solved:** Blocks, Delivery, Gripper, Miconic, Visitall
 - ▷ **Exceptions:** Logistics (60%), Reward (73%), Spanner (73%)
- Why not full coverage then?
 - ▷ Logistics needs role composition, not in C_2 /GNN
 - ▷ Reward/Spanner need to compute distances larger than # of layers
- Fixes:
 - ▷ **Logistics:** add new atoms in states representing role compositions
 - ▷ **Spanner:** add transitive closure of binary predicates
- Results:
 - ▷ **In Logistics and Spanner coverage jumps to 100%**

Deep Reinforcement Learning (DRL)

- Different ways to do it ...
 - ▷ Approximate V - or Q -functions with neural net, do VI or PI
 - ▷ Parameteric policy combined with **policy gradient methods**
- “Deep” means functions/policy represented with deep neural net

Policy Gradient Methods

- Learn parameterized policy to select actions. A value function may be used to learn parameters, but not required for action selection
- Parameter learned with gradients of some performance measure $J(\boldsymbol{\theta})$. Methods seek to maximize performance, so they do gradient ascent
- Methods that learn approximations of policy and value functions called actor–critic: “actor” refers to policy, and “critic” to value function
- To ensure exploration, policy should never become deterministic. If action space is “small”, it is common **softmax actor** from preferences or logits $h(s, a, \boldsymbol{\theta})$:

$$\pi(a|s, \boldsymbol{\theta}) = \frac{e^{h(s,a,\boldsymbol{\theta})}}{\sum_b e^{h(s,b,\boldsymbol{\theta})}}$$

- **Examples:** REINFORCE, Actor-Critic, PPO, ...

REINFORCE

- Classic (perhaps first) well-understood policy gradient method [Williams, 1992]
- Correctness established with **policy gradient theorem** [Sutton & Barto, 2000]
- Updates occur at end of episode

Input: differentiable (parametric) policy $\pi(a|s, \theta)$

Algorithm parameter: step size $\alpha > 0$

- 1: Initialize policy parameters θ (e.g., $\theta = \mathbf{0}$)
- 2: **loop forever** (for each episode):
- 3: Generate episode $S_0, A_0, R_0, \dots, S_{T-1}, A_{T-1}, R_T$ following $\pi(\cdot|\cdot, \theta)$
- 4: **for** each step $t = 0, 1, 2, \dots, T - 1$ **do**
- 5: $G := \sum_{k=t+1}^T \gamma^{k-t-1} R_k$
- 6: $\theta := \theta + \alpha \gamma^k G \nabla \ln \pi(A_t|S_t, \theta)$

Actor-Critic

- Critic $\hat{v}(s, \mathbf{w})$ estimates V -value of state s
- Unlike REINFORCE, parameters updated in the “middle” of episode

Input: differentiable policy $\pi(a|s, \boldsymbol{\theta})$ and state-value function $\hat{v}(s, \mathbf{w})$

Algorithm parameters: step sizes $\alpha > 0$ and $\beta > 0$

```
1: Initialize policy parameters  $\boldsymbol{\theta}$  and state-value parameters  $\mathbf{w}$ 
2: loop forever (for each episode):
3:   Initialize  $S$  (first state of episode)
4:    $I := 1$ 
5:   while  $S$  is not terminal do
6:      $A \sim \pi(\cdot|S, \boldsymbol{\theta})$ 
7:     Take action  $A$ , observe resulting state  $S'$  and reward  $R$ 
8:      $\delta := R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$       (if  $S'$  is terminal,  $\hat{v}(S', \mathbf{w}) \doteq 0$ )
9:      $\mathbf{w} := \mathbf{w} + \beta \delta \nabla \hat{v}(S, \mathbf{w})$ 
10:     $\boldsymbol{\theta} := \boldsymbol{\theta} + \alpha I \delta \nabla \ln \pi(A|S, \boldsymbol{\theta})$ 
11:     $I := \gamma I$ 
12:     $S := S'$ 
```

Actor-Critic Algorithms for Generalized Planning

- Policies map states s into probabilities $\pi(s'|s)$ for successors s'
- Training set is $\{M_i\}$ of deterministic MDPs. M_i sampled using prior p_i , then sample state s in M_i (uniform)
- Action costs set to 1
- When sampled successor s' is goal, parameter update ensure value of s' is zero
- No trajectories sampled, just state transitions

Actor-Critic Algorithms for Generalized Planning

Algorithm 1 Standard Actor-Critic for generalized planning: successor states s' sampled with probability $\pi(s'|s)$.

- 1: **Input:** Training MDPs $\{M_i\}_i$, each with state priors p_i
 - 2: **Input:** Differentiable policy $\pi(s|s')$ with parameter θ
 - 3: **Input:** Diff. value function $V(s)$ with parameter ω
 - 4: **Parameters:** Step sizes $\alpha, \beta > 0$, discount factor γ
 - 5: Initialize parameters θ and ω
 - 6: Loop forever:
 - 7: Sample MDP index $i \in \{1, \dots, N\}$
 - 8: Sample non-goal state S in M_i with probability p_i
 - 9: Sample successor state S' with probability $\pi(S'|S)$
 - 10: Let $\delta = 1 + \gamma V(S') - V(S)$
 - 11: $\omega \leftarrow \omega + \beta \delta \nabla V(S)$ Eq. (14)
 - 12: $\theta \leftarrow \theta - \alpha \delta \nabla \log \pi(S'|S)$ Eq. (8)
 - 13: If S' is a goal state, $\omega \leftarrow \omega - \beta V(S') \nabla V(S')$
-

Algorithm 2 All-Actions Actor-Critic: all successor states considered for updating policy and value functions.

- 1: **Input:** Training MDPs $\{M_i\}_i$, each with state priors p_i
 - 2: **Input:** Differentiable policy $\pi(s|s')$ with parameter θ
 - 3: **Input:** Diff. value function $V(s)$ with parameter ω
 - 4: **Parameters:** Step sizes $\alpha, \beta > 0$, discount factor γ
 - 5: Initialize parameters θ and ω
 - 6: Loop forever:
 - 7: Sample MDP index $i \in \{1, \dots, N\}$
 - 8: Sample non-goal state S in M_i with probability p_i
 - 9: Let $V' = 1 + \gamma \sum_{s' \in N(S)} [\pi(s'|S) V(s')]$
 - 10: Let $b(S) = V' - 1$ be the baseline
 - 11: $\omega \leftarrow \omega + \beta (V' - V(S)) \nabla V(S)$
 - 12: $\theta \leftarrow \theta - \alpha \sum_{s' \in N(S)} [(V(s') - b(S)) \nabla \pi(s'|S)]$
 - 13: If $s' \in N(S)$ is goal state, $\omega \leftarrow \omega - \beta V(s') \nabla V(s')$
-

GNN-Based General Policies

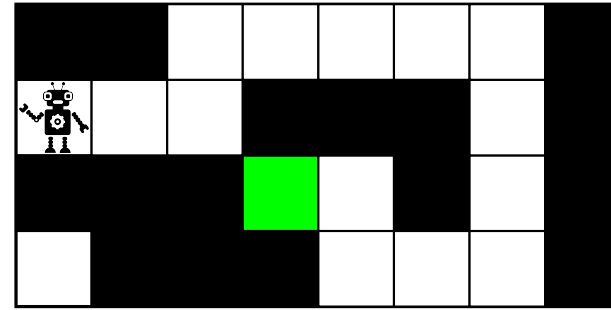
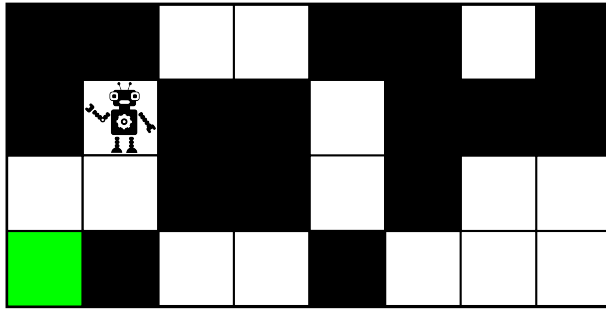
- **Parametric policy:** when at state s , for each successor s' :
 - ▷ Run net to obtain object embeddings for s and s'
 - ▷ “Stack” emb., MLP, sum-aggregate, and another MLP to get (s, s') **logit**
 - ▷ **Softmax policy:** choose edge (s, s') that maximizes

$$\frac{\exp(f(s, s'))}{\sum_{s'' \in \text{Succ}} \exp(f(s, s''))}$$

Results for Actor-Critic Algorithms

Domain (#)	Stochastic Policy			Deterministic Policy			Validation	
	Coverage (%)	L	PQ = PL / OL (#)	Coverage (%)	L	PQ = PL / OL (#)	V_{π^*}	V_{π}
Standard Actor-Critic								
Blocks (23)	23 (100 %)	810	1.00 = 478 / 476 (16)	23 (100 %)	810	1.00 = 478 / 476 (16)	18.60	18.60
Blocks-multiple (26)	26 (100 %)	898	1.00 = 450 / 448 (16)	26 (100 %)	898	1.00 = 450 / 448 (16)	17.57	17.57
Delivery (24)	21 (88 %)	591	1.01 = 544 / 540 (20)	23 (96 %)	701	1.02 = 596 / 586 (21)	16.23	34.46
Grid (14)	5 (36 %)	43	1.00 = 43 / 43 (5)	10 (71 %)	516	2.68 = 356 / 133 (9)	18.13	668.28
Gripper (20)	17 (85 %)	5 031	1.00 = 176 / 176 (4)	16 (80 %)	1 312	1.00 = 176 / 176 (4)	14.89	14.89
Logistics (22)	3 (14 %)	33	1.10 = 33 / 30 (3)	17 (77 %)	37 667	77.4 = 23 839 / 308 (15)	9.04	636.43
Miconic (30)	29 (97 %)	1 438	1.00 = 185 / 185 (10)	29 (97 %)	1 438	1.00 = 185 / 185 (10)	6.42	6.42
Reward (15)	13 (87 %)	2 953	2.38 = 1 328 / 558 (7)	6 (40 %)	655	1.24 = 362 / 292 (4)	22.15	120.34
Spanner (22)	18 (82 %)	682	1.11 = 218 / 197 (7)	18 (82 %)	678	1.11 = 218 / 197 (7)	130.74	130.82
Visitall (24)	24 (100 %)	1 083	1.23 = 762 / 621 (20)	24 (100 %)	1 032	1.12 = 696 / 621 (20)	4.36	4.39
Total (220)	179 (81 %)	13 562	-	192 (87 %)	45 707	-	-	-
All-Actions Actor-Critic								
Blocks (23)	23 (100 %)	806	1.00 = 476 / 476 (16)	23 (100 %)	806	1.00 = 476 / 476 (16)	18.60	18.60
Blocks-multiple (26)	26 (100 %)	902	1.00 = 450 / 448 (16)	26 (100 %)	902	1.00 = 450 / 448 (16)	17.57	17.57
Delivery (24)	23 (96 %)	691	1.00 = 586 / 586 (21)	24 (100 %)	757	1.03 = 652 / 632 (22)	16.23	16.23
Grid (14)	6 (43 %)	71	1.00 = 71 / 71 (6)	11 (79 %)	292	1.50 = 248 / 165 (10)	18.13	84.21
Gripper (20)	20 (100 %)	1 840	1.00 = 176 / 176 (4)	20 (100 %)	1 840	1.00 = 176 / 176 (4)	14.89	14.89
Logistics (22)	0 (0 %)	-	-	8 (36 %)	7 981	63.8 = 7 981 / 125 (8)	9.04	511.76
Miconic (30)	30 (100 %)	1 527	1.00 = 185 / 185 (10)	30 (100 %)	1 527	1.00 = 185 / 185 (10)	6.42	6.42
Reward (15)	7 (47 %)	2 493	1.29 = 442 / 342 (4)	12 (80 %)	1 464	1.21 = 582 / 481 (6)	22.15	167.25
Spanner (22)	15 (68 %)	552	1.10 = 149 / 135 (5)	15 (68 %)	552	1.10 = 149 / 135 (5)	130.74	130.81
Visitall (24)	23 (96 %)	5 670	7.05 = 3 934 / 558 (19)	24 (100 %)	971	1.08 = 671 / 621 (20)	4.36	4.40
Total (220)	173 (79 %)	14 552	-	193 (88 %)	17 092	-	-	-

Grid Navigation with Decoupled Coordinates



- Navigation in grid with obstacles. Decoupled coordinates: $\text{CELL}(x, y)$ and $\text{BLOCKED}(x, y)$, position with $\text{AT}(x, y)$, and $\text{ADJ}(i, i + 1)$ along dimensions
- For computing goal distances (eg V^*), cells (x, y) must “communicate” with neighbors (x, y') and (x', y) . In the plain R-GNN, there must be atoms involving $\{x, y, y'\}$ (similarly, $\{x, x', y\}$). **No such atoms exist**

k -Dimensional WL

- Coloring for k -tuples of vertices rather than single vertices
- For $G = (V, E)$ and tuple $\langle v \rangle \in V^k$, coloring $C_{i+1}(\langle v \rangle)$:

$$C_{i+1}(\langle v \rangle) = \text{RELABEL}(C_i(\langle v \rangle), M_i(\langle v \rangle))$$

where $M_i(\langle v \rangle)$ is the multiset of colored tuples

$$M_i(\langle v \rangle) = \{ \{ C_i(\phi_1(\langle v \rangle, w)), \dots, C_i(\phi_k(\langle v \rangle, w)) \} \mid w \in V \}$$

function $\phi_j(\langle v \rangle, w)$ replaces j -th component of $\langle v \rangle$ with w

- Logical characterization with logic C^k (at most k vars, and counting quant.)
- Coloring computed in $\mathcal{O}(|V|^k)$ time
- Expressivity increases with k : $k=2$ is significant jump over $k=1$; e.g.

$$(R \bowtie S)(x, y) \equiv \exists z [R(x, z) \wedge S(z, y)]$$

Parametric Extended R-GNN $[t]$ [Stahlberg et al., 2025]

- Built on top of R-GNN architecture: $\text{R-GNN}[t](S, O) = \text{R-GNN}(A_t(S), O^2)$
 - ▷ t is non-negative integer parameter: as $t \rightarrow \infty$ expressivity approaches 2-WL
 - ▷ $A_t(S) = A_0(S) \cup \Delta_t(S)$ transforms and extend atoms in S

$$A_0(S) = \{p(\langle w \rangle^2) \mid p(w) \in S\}$$

$$\Delta_t(S) = \{\Delta(\langle o, o' \rangle, \langle o', o'' \rangle, \langle o, o'' \rangle) \mid \langle o, o' \rangle, \langle o', o'' \rangle \in R_t\}$$

$$\langle w \rangle^2 = \langle (o_1, o_1), (o_1, o_2), \dots, (o_n, o_n) \rangle \text{ for } \langle w \rangle = \langle o_1, \dots, o_n \rangle$$

where relation R_t is given by

$$\langle o, o' \rangle \in R_t \text{ iff } \begin{cases} o \text{ and } o' \text{ are both in some atom in } S & t = 1 \\ \exists o'' [\{\langle o, o'' \rangle, \langle o'', o' \rangle\} \subseteq R_{t-1}] & t > 1 \end{cases}$$

- ▷ Final readout $V(S) = \sum_o \mathbf{f}(\langle o, o \rangle)$
- Controlled and relational version of 2-WL:
 - ▷ Embeddings for object pairs by replacing atoms in S with $A_0(S)$
 - ▷ Messages among pairs mediated by new Δ atoms (and MLP_Δ)
 - ▷ t controls number of Δ -atoms via depth of compositions

Results

Domain	Model	Coverage (%)	Plan Length		
			Total	Median	Mean
Blocks-s	R-GNN	17 / 17 (100 %)	674	38	39
	R-GNN[0]	17 / 17 (100 %)	670	36	39
	R-GNN[1]	17 / 17 (100 %)	684	36	40
	R-GNN ₂	14 / 17 (82 %)	922	35	65
Blocks-m	R-GNN	22 / 22 (100 %)	868	40	39
	R-GNN[0]	22 / 22 (100 %)	830	39	37
	R-GNN[1]	22 / 22 (100 %)	834	39	37
	R-GNN ₂	22 / 22 (100 %)	936	39	42
Gripper	R-GNN	18 / 18 (100 %)	4,800	231	266
	R-GNN[0]	18 / 18 (100 %)	1,764	98	98
	R-GNN[1]	11 / 18 (61 %)	847	77	77
	R-GNN ₂	18 / 18 (100 %)	1,764	98	98
Miconic	R-GNN	20 / 20 (100 %)	1,342	67	67
	R-GNN[0]	20 / 20 (100 %)	1,566	71	78
	R-GNN[1]	20 / 20 (100 %)	2,576	71	128
	R-GNN ₂	20 / 20 (100 %)	1,342	67	67
Visitall	R-GNN	18 / 22 (82 %)	636	29	35
	R-GNN[0]	21 / 22 (95 %)	1,128	35	53
	R-GNN[1]	22 / 22 (100 %)	886	35	40
	R-GNN ₂	20 / 22 (91 %)	739	33	36

Domain	Model	Coverage (%)	Plan Length		
			Total	Median	Mean
Grid	R-GNN	9 / 20 (45 %)	109	11	12
	R-GNN[0]	12 / 20 (60 %)	177	11	14
	R-GNN[1]	15 / 20 (75 %)	209	13	13
	R-GNN ₂	10 / 20 (50 %)	124	11.5	12
Logistics	R-GNN	10 / 20 (50 %)	510	51	51
	R-GNN[0]	9 / 20 (45 %)	439	48	48
	R-GNN[1]	20 / 20 (100 %)	1,057	52	52
	R-GNN ₂	15 / 20 (75 %)	799	52	53
Rovers	R-GNN	9 / 20 (45 %)	2,599	280	288
	R-GNN[0]	14 / 20 (70 %)	2,418	153	172
	R-GNN[1]	14 / 20 (70 %)	1,654	55	118
	R-GNN ₂	11 / 20 (55 %)	2,225	239	202
Vacuum	R-GNN	20 / 20 (100 %)	4,317	141	215
	R-GNN[0]	20 / 20 (100 %)	183	9	9
	R-GNN[1]	20 / 20 (100 %)	192	9	9
	R-GNN ₂	20 / 20 (100 %)	226	9	11
Visitall-xy	R-GNN	5 / 20 (25 %)	893	166	178
	R-GNN[0]	15 / 20 (75 %)	1,461	84	97
	R-GNN[1]	20 / 20 (100 %)	1,829	83	91
	R-GNN ₂	19 / 20 (95 %)	2,428	116	127

- R-GNN₂: all atoms $\Delta(\langle o, o' \rangle, \langle o', o'' \rangle, \langle o, o'' \rangle)$

Learning STRIPS Models

STRIPS Models

- In all case before, we need the domain's (STRIPS) model:
 - ▷ **Symbolic:** model seeds the feature pool that make up the state abstraction
 - ▷ **Neural:** input to relational net is symbolic state made of gounded atoms
- **Task:** basically, to learn the lifted action schemas. This may include learning the involved predicates
- Different approaches for learning models from traces $s_0, a_0, s_1, a_1, \dots$, depending on the **information** contained in traces:
 - ▷ **Full states + full labels:** **too much, learning problem is trivial!**
 - ▷ **Full states + partial labels:** predicates and objects given, still non-trivial
 - ▷ **No states + full labels:** objects given, must learn predicates and schemas
 - ▷ **No states + partial labels:** most difficult, no objects, predicates, or schemas

SIFT: No States + Full Labels (Symbolic Approach)

- Recent and efficient learning approach by [Gösgens et al., 2025]
- (Action) traces of form $\tau = a_0, a_1, \dots$ where $a_i = \text{label}(o_1, o_2, \dots, o_m)$
 - ▷ Each trace associated with implicit states $\tau_0, \tau_1, \dots$, but no assumption about them
 - ▷ Additional information about **state equality** (inter-trace and intra-trace) may be provided
- Dual representation:
 - ▷ **(Standard) action-first:** action $a(x_1, \dots, x_m)$ comes with $Pre(a)$, $Add(a)$ and $Del(a)$
 - ▷ **Atom-first:** atom $p(x_1, \dots, x_k)$ comes with list of actions that “refer to it”
- Examples:
 - ▷ **Gripper:** (at ?obj ?room)
 - Prec: pick(?obj, ?room, ?gripper)
 - Added: drop(?obj, ?room, ?gripper)
 - Removed: pick(?obj, ?room, ?gripper)
 - ▷ **Blocks3ops:** (on ?x ?y)
 - Prec: unstack(?x, ?y), move(?x, ?y, ?z)
 - Added: stack(?x, ?y), move(?x, ?z, ?y)
 - Removed: unstack(?x, ?y), move(?x, ?y, ?z)

Action Patterns and Features

- Feature is pair $f = (k, B)$ where $k > 0$ and B is set of “patterns” $\text{label}(i_1, \dots, i_k)$ such that $1 \leq i_j \leq \text{arity}(\text{label})$ for $j = 1, 2, \dots, k$
 - ▷ **Gripper:** $(1, \{\text{move}(1), \text{move}(2)\})$, $(2, \{\text{pick}(1, 2), \text{drop}(1, 2)\})$, $\dots$
 - ▷ **Blocks3ops:** $(2, \{\text{stack}(1, 2), \text{unstack}(1, 2), \text{move}(1, 2), \text{move}(1, 3)\})$

Feature represents assumption about domain: it contains predicate f of arity k

- **Signed feature** is feature $\bar{f} = (k, B)$ where patterns in B are **positive** or **negative**
 - ▷ **Positive pattern** $a(i_1, \dots, i_k)$ means action $a(o_1, \dots, o_m)$ **adds** $f(o_{i_1}, \dots, o_{i_k})$
 - ▷ **Negative pattern** $a(i_1, \dots, i_k)$ means action $a(o_1, \dots, o_m)$ **removes** $f(o_{i_1}, \dots, o_{i_k})$
- Signed feature $\bar{f} = (k, B)$ is **consistent** in $\tau = \langle a_\ell \rangle_\ell$:
 - ▷ If a_i, a_j are **consecutive occurrences** of patterns in B , they have opposite signs
- Signed feature $\bar{f} = (k, B)$ is **consistent** in $\tau = \langle a_\ell \rangle_\ell$ and $\tau' = \langle b_\ell \rangle_\ell$:
 - ▷ If a_i, b_j are **final occurrences before** τ_k and $\tau'_{k'}$ with $\tau_k = \tau'_{k'}$, they have equal signs
 - ▷ If a_i, b_j are **first occurrences after** τ_k and $\tau'_{k'}$ with $\tau_k = \tau'_{k'}$, they have equal signs

Learning in SIFT

- Input traces fix vocabulary (schemas + arities) which implies finite features
- **Learning:** find maximal set of signed features consistent with input traces
 - ▷ **Crucial: features do not interact!**
- **Theorem.** Finding (and deciding) consistent assignment of signs to given feature can be done in linear time (2-CNF)
- Learning algorithm:

Input: set T of traces possibly augmented with state equalities

Output: maximal set $F(T)$ of signed features consistent with T and state equalities

```
1:  $F(T) := \emptyset$ 
2: for each feature  $f = (k, B)$  do
3:   if there is consistent signed  $\bar{f}$  then
4:      $F(T) := F(T) \cup \{\bar{f}\}$ 
```

From Consistent Signed Features to STRIPS Model

- Each consistent signed feature $\bar{f} = (k, B)$ becomes predicate of arity k
- Consider $\bar{f} = (k, B)$, trace $\tau = \langle s_\ell \rangle_\ell$ in T , and **action grounding**:

$$A_{f,\tau}(o) = \{a_i(o') \in \tau \mid a[t] \in B, o = t[o']\}$$

- **Assume hidden domain is well-formed (ie. each effect changes state):**
 - ▷ If pattern $a_i(o') \in A_{f,\tau}(o)$ is **positive**, then say $s_i \not\vdash f(o)$
 - ▷ If pattern $a_i(o') \in A_{f,\tau}(o)$ is **negative**, then say $s_i \vdash f(o)$
- Learned domain D_T with lifted actions $a(x)$ as in traces, and predicates $f(x)$ as in signed features $\bar{f} = (k, B)$ in $F(T)$:
 - ▷ Effects $f(t[x])$ of $a(x)$ given by sign of pattern $a[t]$ in B
 - ▷ **Precondition** $f(t[x])$ for $a(x)$ iff $\forall(\tau \in T) [a_i(o') \in \tau \implies s_i \vdash f(t[o'])]$
 - ▷ Similar for negative preconditions
- Static predicates cannot be inferred; for them need negative traces

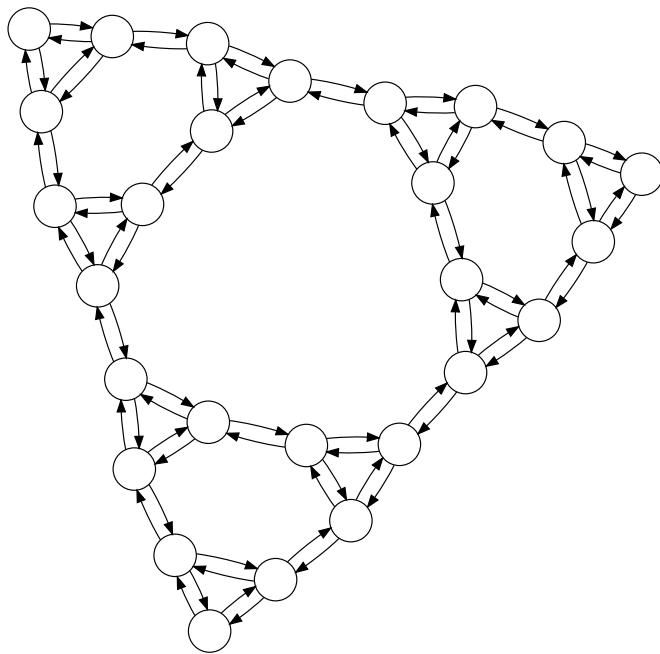
SIFT Results

Domain	# O	# P	# F	Full Graphs				Partial Graphs				Traces			
				# F_a	# E	Time	Verif	# F_a	# E	Time	Verif	# F_a	# E	Time	Verif
blocks3	6	3	1220	5.0	21300	51 s	100%	5.0	81	28 s	100%	5.0	65	29 s	100%
blocks4	7	5	93	9.0	186578	587 s	100%	9.0	86	17 s	100%	9.0	85	17 s	100%
delivery	13	3	62	5.0	57888	183 s	100%	5.0	601	105 s	100%	5.0	350	100 s	100%
driverlog	11	4	560	13.0	63720	700 s	100%	13.0	1201	777 s	100%	13.0	350	683 s	100%
ferry	10	4	31	4.0	156250	347 s	100%	4.0	251	31 s	100%	4.0	170	24 s	100%
grid	14	6	290	7.0	99863	546 s	100%	7.0	10001	37 s	100%	29.7	10000	36 s	0%
grid_lock	14	6	1042	7.0	152040	1678 s	100%	7.0	500	123 s	100%	7.0	800	117 s	100%
gripper	12	4	43	6.0	95680	212 s	100%	6.0	230	64 s	100%	6.0	250	306 s	100%
hanoi	12	2	134	4.0	59046	2164 s	100%	4.0	50	25 s	100%	4.0	25	24 s	100%
logistics	18	2	212	7.0	648648	12866 s	100%	7.0	5003	1403 s	100%	7.0	350	1304 s	100%
miconic	10	3	99	8.0	127008	376 s	100%	8.0	46	33 s	100%	8.0	60	36 s	100%
npuzzle	14	2	912	26.0	483840	11598 s	100%	26.0	130	312 s	100%	26.0	200	311 s	100%
sokoban	16	2	352	3.0	26834	231 s	100%	3.0	10002	195 s	100%	126.2	10000	220 s	4%
sokoban_pull	16	2	20740	3.0	66328	401 s	100%	3.0	300	77 s	100%	3.0	300	123 s	100%

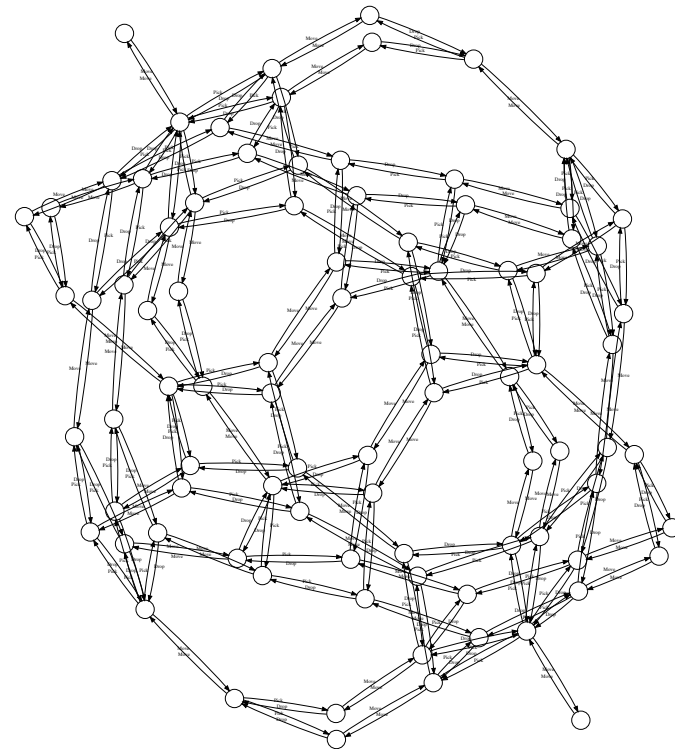
Table 1: Results table. For each domain: number of objects $\#O$ in training instance, number of non-static predicates $\#P$, number of features to test $\#F$, and for each input (full and partial graphs, plain traces): avg. number of admissible features $\#F_a$, avg. number of edges in input graphs $\#E$ (trace length for traces), avg. total time (data generation, learning, verification), and ratio of successful verification tests (Verif). Averages over 25 runs except for full graphs (not sampled)

Black-Box States and Action Labels Only

- [B. & Geffner, 2020; Rodriguez et al, 2021] (SAT; ASP, noisy + inc. graphs)
- No information about objects, states or action arguments
- Input is transition graph with opaque vertices and labels only carry action name



Towers of Hanoi (27s, 78e, 1 label)



Gripper (88s, 280e, 3 labels)

Learning Formulation

- **Task:** Find STRIPS model + instance data (objects + initial state) such that input graph(s) **isomorphic** to model graph(s)
- Two-level combinatorial search:
 - ▷ **Parameter space:** bound on # objects, predicates, precs, effects, and arities
 - ▷ **Model space:** search model with given bounds that yields isomorphic graphs
- Solution reveals:
 - ▷ Model: lifted predicates and actions (including static preconditions)
 - ▷ Objects and denotations for states in input graph(s)
 - ▷ Existent and non-existent transitions (edges) are explicated by model
- Hanoi input yields following model that works for any number of disks and pegs:

```
MoveDisk(disk, to, from):  
  Static: NEQ(disk, to), NEQ(disk, from), NEQ(to, from), -BIGGER(disk, to)  
  Prec:   -p(to, disk), -p(from, from), p(disk, disk), p(to, to), p(from, disk)  
  Effect: -p(to, to), -p(from, disk), p(to, disk), p(from, from)
```

$p(x, y)$ denotes $on(x, y)$ when $x \neq y$; else $clear(x)$

Example for Gripper

Learned model that is correct for any $\#$ rooms, balls, and grippers:

`Move(x, to, from):`

Static: `NEQ(x, from), NEQ(to, from), -B1(x, to), -B2(x, x), B1(from, x)`

Prec: `-Nat(from), Nat(x), Nat(to)`

Effect: `-Nat(to), Nat(from)`

`Drop(gripper, ball, room):`

Static: `NEQ(gripper, room), -B2(room, ball), B1(gripper, gripper)`

Prec: `-Nat(room), -at-hold(room, ball), Nfree(gripper), at-hold(gripper, ball)`

Effect: `-Nfree(gripper), -at-hold(gripper, ball), at-hold(room, ball)`

`Pick(gripper, room, ball):`

Static: `NEQ(gripper, room), -B2(room, ball), B1(gripper, gripper)`

Prec: `-Nat(room), -Nfree(gripper), -at-hold(gripper, ball), at-hold(room, ball)`

Effect: `-at-hold(room, ball), Nfree(gripper), at-hold(gripper, ball)`

- ▷ $at\text{-}hold(x, ball)$ is $at(x, ball)$ if x is room, and $hold(x, ball)$ if x is gripper
- ▷ Static $B1(x_1, x_2)$ encodes x_1 is gripper if $x_1 = x_2$, or x_1 and x_2 are rooms otherwise
- ▷ $B2(x_1, x_2)$ encodes pairs (x_1, x_2) that are not a room and a ball

Glimpse at ASP Formulation

```
% Constants
#const max_predicates=5.
#const max_effects=6.           % for each action
#const max_precs=6.             % for each action

% Actions, predicates and objects
action(L) :- LABEL(E,L). { a_arity(A,1..3) } = 1 :- action(A).
pred(1..max_predicates). { p_arity(P,1..2) } = 1 :- pred(P).
object(1..num_objects).    % num_objects is set by command line

% Tuples of variables for lifted prec and effects
argtuple((V1, ),1) :- V1=1..3.
argtuple((V1,V2),2) :- V1=1..3, V2=1..3.

% Generate lifted order prec and effects
{ prec(A,(P,T),0..1): p_arity(P,AR), argtuple(T,AR) } max_precs    :- action(A).
1 { eff(A,(P,T),0..1): p_arity(P,AR), argtuple(T,AR) } max_effects :- action(A).

% Check that variables mentioned in prec and effects are action arguments
:- eff(A,(_, (V, )),_), a_arity(A,ARITY), ARITY < V.
:- eff(A,(_, (V, _)),_), a_arity(A,ARITY), ARITY < V.
:- eff(A,(_, (_, V)),_), a_arity(A,ARITY), ARITY < V.
:- prec(A,(_, (V, )),_), a_arity(A,ARITY), ARITY < V.
...

% Tuples of objects for grounding atoms, prec and effects
objtuple((O1, ),1) :- object(O1).
objtuple((O1,O2 ),2) :- object(O1), object(O2).
objtuple((O1,O2,O3),3) :- object(O1), object(O2), object(O3).
```

```

% Generation of STRIPS state for every node
{ val((P,0),N,0..1) } = 1 :- p_arity(P,ARITY), objtuple(0,ARITY), NODE(N).

% Check nodes correspondence (different nodes are different states)
:- NODE(N1), NODE(N2), N1 < N2, val((P,T),N2,V) : val((P,T),N1,V).

% Map tuples of variables and objects to tuples of objects
map((1,) ,(01,) ,(01,) ) :- objtuple((01,) ,1).
map((1,) ,(01,02) ,(01,) ) :- objtuple((01,02) ,2).
...
map((3,2) ,(01,02,03) ,(03,02)) :- objtuple((01,02,03) ,3).
map((3,3) ,(01,02,03) ,(03,03)) :- objtuple((01,02,03) ,3).

% Ground action A(001) is applicable in node N
appl(A,001,N) :- action(A), a_arity(A,ARITY), objtuple(001,ARITY), NODE(N),
                val((P,002),N,V) : prec(A,(P,T),V), map(T,001,002).

% Check edges correspondence
{ next(A,00,N1,N2) : LABEL((N1,N2),A) } = 1 :- appl(A,00,N1).
:- LABEL((N1,N2),A), { next(A,00,N1,N2) } != 1.

% Check nodes correspondence (effects and inertia)
caused(N1,N2,(P,002)) :- eff(A,(P,T),V), next(A,001,N1,N2), map(T,001,002).
:- eff(A,(P,T),V), next(A,001,N1,N2), map(T,001,002), val((P,002),N2,1-V).
:- LABEL((N1,N2),_), val(K,N1,V), val(K,N2,1-V), not caused(N1,N2,K).

```

Results on Complete and Noise-Free Graphs

Domain	#obj	#mod	Time to find model		Ver	Opt
			First	Best		
blocks1-3	3	8	35.08	93.85	✗	✓
blocks1-4	4	0	—	—	—	—
blocks2-3	3	15	3.86	10.84	✗	✓
blocks2-4	4	26	550.1	2,564.2	✓	✓
grid-v0-3x4	2	13	2.9	13.9	✓	✓
grid-v0-3x4	3	14	5.5	23.1	✓	✓
grid-v0-3x4	4	20	3.2	24.1	✓	✓
grid-v1-3x4	3	13	1.0	105.2	✓	✓
grid-v1-3x4	4	12	1.2	12.4	✓	✓
gripper-2	4	37	356.3	7,200.0	✓	✗
gripper-3	4	1	6,540.2	7,200.0	✓	✗
hanoi-3x3	6	6	664.9	7,200.1	✓	✗

Columns show instance name, number of objects found, times for first and best solution found, and whether best solution verified and was shown optimal. Column ‘#mod’ is number of models found in way to best model. Times are in seconds.

Results on Incomplete and Noisy Graphs

Domain	#obj	p	n	v	opt
blocks2-4	4	20	10	10	3
blocks2-4	4	60	10	9	1
blocks2-4	4	100	10	10	0
grid-v0-3x4	4	20	10	0	10
grid-v0-3x4	4	60	10	10	10
grid-v0-3x4	4	100	10	10	10
grid-v1-3x4	4	20	10	2	10
grid-v1-3x4	4	60	10	9	10
grid-v1-3x4	4	100	10	9	10
gripper-2	4	20	10	0	9
gripper-2	4	60	10	4	0
gripper-2	4	100	10	6	0
gripper-3	4	20	2	1	0
gripper-3	4	60	1	0	0
gripper-3	4	100	1	1	0
hanoi-3x3	6	20	10	0	5
hanoi-3x3	6	60	10	10	0
hanoi-3x3	6	100	9	9	0

Partial graphs

Domain	#obj	q	n	v	opt
blocks2-4	4	20	2	2	0
blocks2-4	4	60	9	6	0
blocks2-4	4	100	10	10	0
grid-v0-3x4	4	20	10	10	10
grid-v0-3x4	4	60	10	10	10
grid-v0-3x4	4	100	10	10	10
grid-v1-3x4	4	20	10	6	10
grid-v1-3x4	4	60	10	5	10
grid-v1-3x4	4	100	10	9	10
gripper-2	4	20	10	5	0
gripper-2	4	60	10	7	0
gripper-2	4	100	10	6	0
gripper-3	4	20	0	0	0
gripper-3	4	60	0	0	0
gripper-3	4	100	1	1	0
hanoi-3x3	6	20	0	0	0
hanoi-3x3	6	60	6	6	0
hanoi-3x3	6	100	9	9	0

Noisy graphs

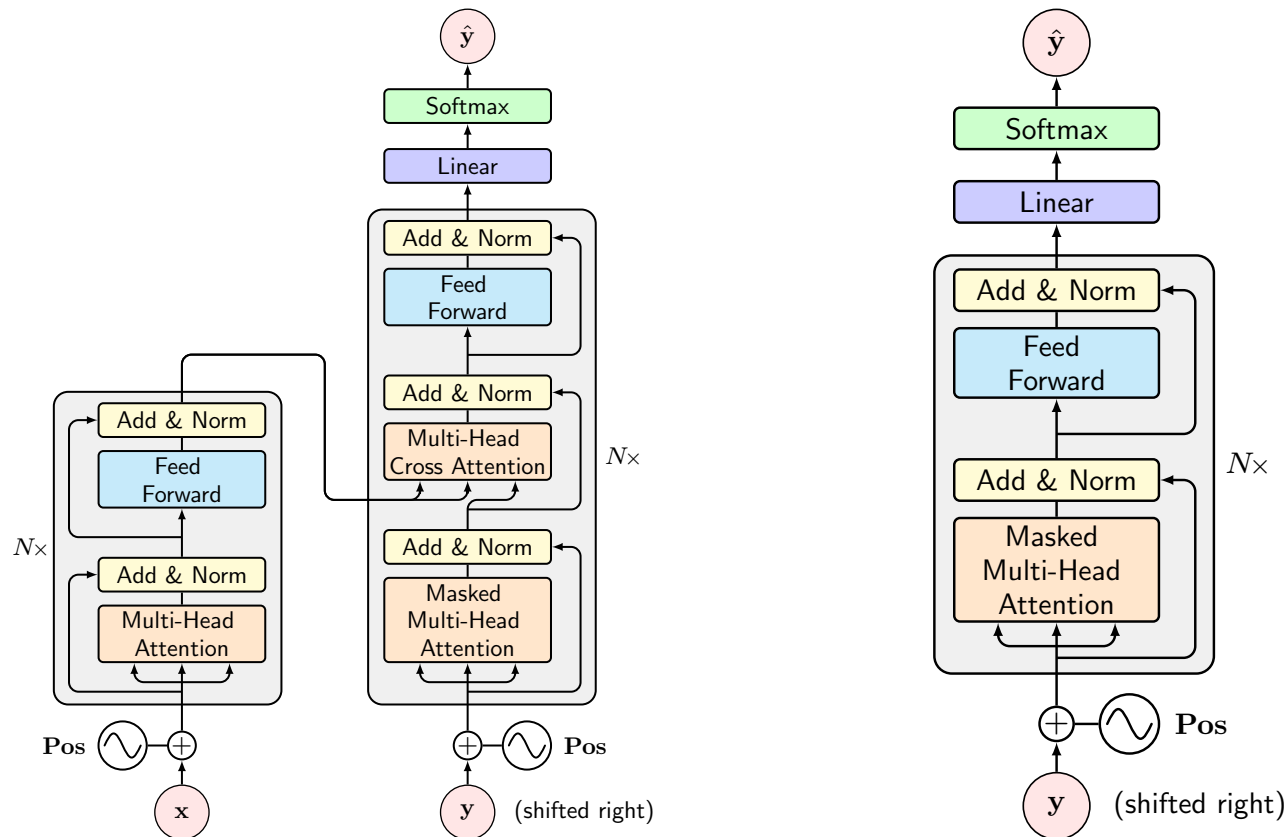
Column p is percentage of sample edges in 10 runs, q is percentage of edges that are not noisy in 10 runs, n how many solutions were found in those runs, v how many of them verify, and opt how many were proved optimal.

Neural Learning of Propositional STRIPS

- Transformer-based learning of propositional models [Núñez-Molina et al., 2026]
- Focus is propositional rather than lifted STRIPS
- **Two architectures:** STRIPS transformer and Stick-Breaking (SB) transformer
- Learn from applicable (positive) and non-applicable (negative) **action traces**
- Experiments show both architectures generalize compositionally over long horizons and support planning using off-the-shelf planners

Transformers [Vaswani et al., 2017]

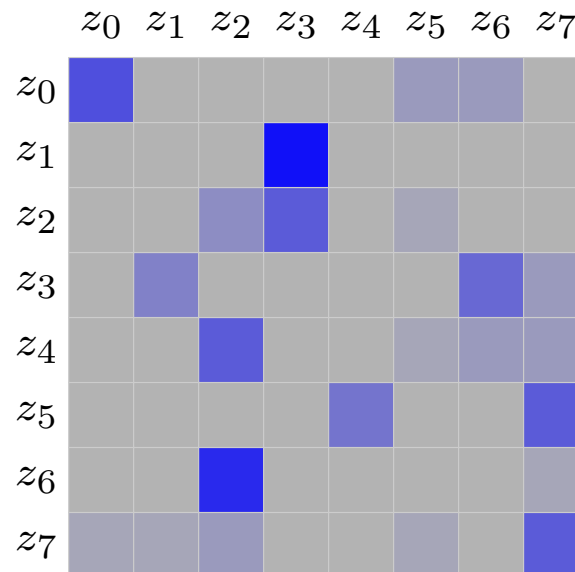
- Sequence-to-sequence (seq2seq) computation
- Based on **self-attention** (originally, scaled dot-product self-attention) to process sequences in parallel and enable long-range dependencies between tokens



- ▷ **Encoder-Decoder:** $P(y_1 \dots y_N | \text{input}) = \prod_i P(y_i | y_{\leq i}, \text{input})$
- ▷ **Decoder-Only:** $P(y_1 \dots y_N) = \prod_i P(y_i | y_{\leq i})$

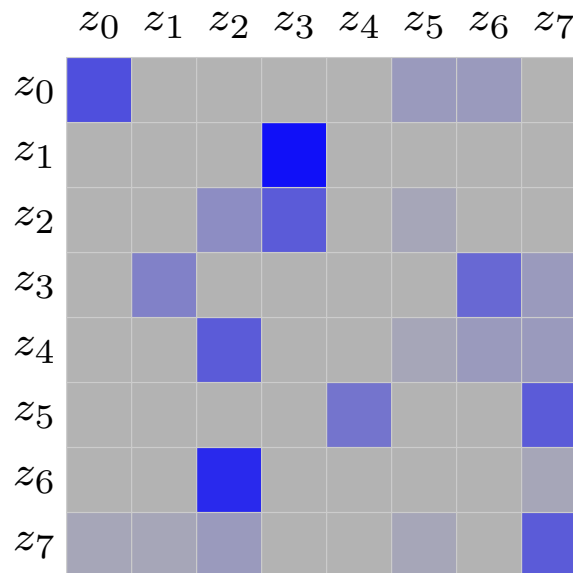
Attention

- **Scaled dot-product attention:** sequence $z_1 \dots z_N$; **head** (Q, K, V) ; token i
 - ▷ $\alpha_{ij} = \text{Softmax} \left(\frac{\text{score}(i, j)}{\sqrt{d}} \right)$ where $\text{score}(i, j) = q_i^\top k_j$, $q_i = z_i \cdot Q$ and $k_j = z_j \cdot K$
 - ▷ $\text{attention}(i) = \sum_{j \leq i} \alpha_{ij} v_j$ where $v_j = z_j \cdot V$, $\alpha_{ij} \geq 0$ and $\sum_{j \leq i} \alpha_{ij} = 1$

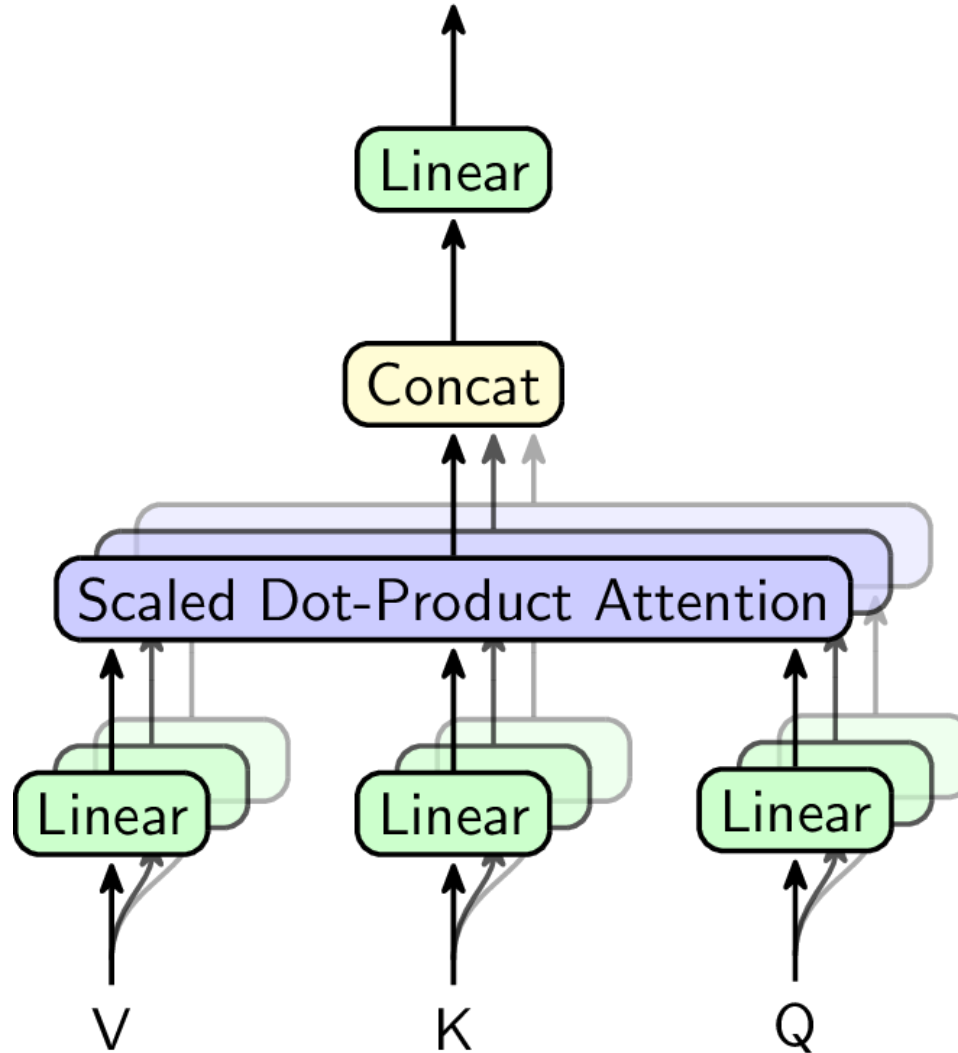


Attention

- **Scaled dot-product attention:** sequence $z_1 \dots z_N$; **head** (Q, K, V) ; token i
 - ▷ $\alpha_{ij} = \text{Softmax} \left(\frac{\text{score}(i, j)}{\sqrt{d}} \right)$ where $\text{score}(i, j) = q_i^\top k_j$, $q_i = z_i \cdot Q$ and $k_j = z_j \cdot K$
 - ▷ $\text{attention}(i) = \sum_{j \leq i} \alpha_{ij} v_j$ where $v_j = z_j \cdot V$, $\alpha_{ij} \geq 0$ and $\sum_{j \leq i} \alpha_{ij} = 1$
- **Stick-breaking attention:**
 - ▷ $\alpha_{ij} = \beta_{ij} \prod_{k=j+1}^i (1 - \beta_{ik})$ where $\beta_{ij} = \text{sigmoid}(q_i^\top k_j)$ (break-point prob.)



Multi-Head Attention



Learning Task

- For propositional STRIPS model $M = (F, A)$ and trace $\tau = (a_0, a_1, \dots, a_{n-1})$:
 - ▷ **Trace τ is positive** iff for each $i = 0, \dots, n - 1$ and precondition p of a_i :
 - last action before a_i that affects p makes p true, or
 - no action before a_i affects p
 - ▷ **Trace τ is negative** iff it is not positive
- Boolean function f_M maps traces τ into $\{0, 1\}$ whether negative or positive
- **Learning task:** Given set T of positive and negative traces from hidden domain $M = (F, A)$, learn Boolean function f such that $f(\tau) = f_M(\tau)$ for $\tau \in T$

Stick-Breaking Transformer

- Stick-breaking transformer consists of:
 - ▷ Learned action tokens (embeddings)
 - ▷ L layers with H attention heads per layer. Each implements stick-breaking attention with strict future masking, followed by additional transformations
 - ▷ Final linear layer with sigmoid activation
- Setup actions:
 - ▷ **Action init-false:** sets all atoms to false
 - ▷ **Action init- p :** sets p to true
 - ▷ **Action test- p :** does nothing but has precondition p
 - ▷ Traces in training extended with prefix and suffix of setup actions
 - ▷ SB transformer trained to predict applicability of domain and setup actions
 - ▷ Loss $\mathcal{L}(\theta) = \frac{1}{|T|} \sum_{\tau \in T} \frac{1}{|\tau|} \sum_{0 \leq i < |\tau|} \ell(y_\theta(i), z_i)$ where $y_\theta(i)$ is the output for sequence $\tau_{\leq i}$, $z_i \in \{0, 1\}$ is label, and ℓ is “focal loss”:

$$\ell(y, z) = -\alpha z(1 - y)^\gamma \log y - (1 - \alpha)(1 - z)y^\gamma \log(1 - y)$$

variant of binary cross-entropy with parameters $\alpha \in [0, 1]$ (relative importance of positive/negative examples) and $\gamma \geq 0$ (reduces contribution of well-classified examples, focusing learning on harder cases); values $\alpha = 0.999$ and $\gamma = 1$

Results

Blocksworld				
Model	Train	Test	Plan (M_S)	Plan (M_L)
SB	1.0 (1.0)	.998 (.999)	1.0 (1.0)	1.0 (1.0)
Sinusoidal	.994 (.998)	.237 (.250)	1.0 (1.0)	.00 (.00)
RoPE	.998 (.999)	.363 (.377)	1.0 (1.0)	.00 (.00)
Ferry				
Model	Train	Test	Plan (M_S)	Plan (M_L)
SB	1.0 (1.0)	.999 (.998)	1.0 (1.0)	1.0 (1.0)
Sinusoidal	.999 (.999)	.258 (.258)	1.0 (1.0)	.00 (.00)
RoPE	1.0 (1.0)	.530 (.541)	1.0 (1.0)	.25 (.04)
Npuzzle				
Model	Train	Test	Plan (M_S)	Plan (M_L)
SB	1.0 (1.0)	.997 (.997)	1.0 (1.0)	1.0 (1.0)
Sinusoidal	.999 (.999)	.253 (.253)	1.0 (1.0)	.01 (.01)
RoPE	.998 (.999)	.402 (.445)	1.0 (1.0)	.02 (.03)
Maze				
Model	Train	Test	Plan (M_S)	Plan (M_L)
SB	1.0 (1.0)	.958 (.967)	.98 (.98)	.93 (.93)
Sinusoidal	1.0 (1.0)	.510 (.506)	.98 (.98)	.00 (.00)
RoPE	1.0 (1.0)	.578 (.562)	.98 (.98)	.00 (.00)
Logistics				
Model	Train	Test	Plan (M_S)	Plan (M_L)
SB	1.0 (1.0)	.987 (.993)	1.0 (1.0)	1.0 (1.0)
Sinusoidal	.999 (1.0)	.281 (.284)	1.0 (1.0)	.00 (.00)
RoPE	.999 (.999)	.412 (.402)	1.0 (1.0)	.32 (.06)

Wrap Up

- Generalized planning is concrete task where different threads meets:
 - ▷ Abstract representations for states, transitions, and paths
 - ▷ Logic, coloring algorithms, etc
 - ▷ Symbolic and neural methods for learning general policies
 - ▷ Symbolic and neural methods for learning STRIPS models
- Temporal abstractions (sketches) **not covered** here:
 - ▷ Rule $r = \{C\} \mapsto \{E\}$ compatible with $s_0, \dots, s_k$ if $s_0 \models C$ and (s_0, s_k) is compatible with r
 - ▷ Theoretical notion of planning width links complexity of finding compatible path with STRIPS domain
- Briefly covered some related work, references provide further links

References

- [Aineto et al., 2019] Aineto, D., Celorrio, S. J., and Onaindia, E. (2019). Learning action models with minimal observability. *Artificial Intelligence*, 275:104–137.
- [Asai and Muise, 2020] Asai, M. and Muise, C. (2020). Learning neural-symbolic descriptive planning models via cube-space priors: The voyage home (to strips). In *Proc. IJCAI*.
- [Barceló et al., 2020] Barceló, P., Kostylev, E. V., Monet, M., Pérez, J., Reutter, J., and Silva, J. P. (2020). The logical expressiveness of graph neural networks. In *ICLR*.
- [Bonet et al., 2017] Bonet, B., De Giacomo, G., Geffner, H., and Rubin, S. (2017). Generalized planning: Non-deterministic abstractions and trajectory constraints. In *Proc. IJCAI*, pages 873–879.
- [Bonet et al., 2019] Bonet, B., Francès, G., and Geffner, H. (2019). Learning features and abstract actions for computing generalized plans. In *Proc. AAAI*, pages 2703–2710.
- [Bonet and Geffner, 2018] Bonet, B. and Geffner, H. (2018). Features, projections, and representation change for generalized planning. In *Proc. IJCAI*, pages 4667–4673.
- [Bonet and Geffner, 2020] Bonet, B. and Geffner, H. (2020). Learning first-order symbolic representations for planning from the structure of the state space. In *Proc. ECAI*, pages 2322–2329.
- [Bonet and Geffner, 2024] Bonet, B. and Geffner, H. (2024). General policies, subgoal structure, and planning width. *JAIR*, 80:475–516.
- [Bonet and Geffner, 2025] Bonet, B. and Geffner, H. (2025). Learning general policies from examples. In *Proc. KR*.
- [Cai et al., 1992] Cai, J.-Y., Fürer, M., and Immerman, N. (1992). An optimal lower bound on the number of variables for graph identification. *Combinatorica*, 12(4):389–410.
- [Chen et al., 2026] Chen, D. Z., Hofmann, T., Klassen, T. Q., and McIlraith, S. A. (2026). Satisficing and Optimal Generalised Planning via Goal Regression. In *Proc. AAAI*.
- [Cresswell and Gregory, 2011] Cresswell, S. N. and Gregory, P. (2011). Generalised domain model acquisition from action traces. In *Twenty-First International Conference on Automated Planning and Scheduling*.

- [De Giacomo et al., 2016] De Giacomo, G., Murano, A., Rubin, S., and Di Stasio, A. (2016). Imperfect-information games and generalized planning. In *IJCAI*, pages 1037–1043.
- [Francès et al., 2021] Francès, G., Bonet, B., and Geffner, H. (2021). Learning general planning policies from small examples without supervision. In *Proc. AAAI*, pages 11801–11808.
- [Garg et al., 2020] Garg, S., Bajpai, A., and Mausam (2020). Symbolic network: generalized neural policies for relational mdps. In *International Conference on Machine Learning*, pages 3397–3407.
- [Grohe, 2020] Grohe, M. (2020). The logic of graph neural networks. In *Proc. of the 35th ACM-IEEE Symp. on Logic in Computer Science*.
- [Groshev et al., 2018] Groshev, E., Goldstein, M., Tamar, A., Srivastava, S., and Abbeel, P. (2018). Learning generalized reactive policies using deep neural networks. In *Proc. ICAPS*.
- [Groshev et al., 2017] Groshev, E., Tamar, A., Srivastava, S., and Abbeel, P. (2017). Learning generalized reactive policies using deep neural networks. *arXiv preprint arXiv:1708.07280*.
- [Gösgens et al., 2025] Gösgens, J., Jansen, N., and Geffner, H. (2025). Learning lifted strips models from action traces alone: A simple, general, and scalable solution. In *Proc. ICAPS*.
- [Gösgens et al., 2026] Gösgens, J., Jansen, N., and Geffner, H. (2026). Learning lifted action models from traces with minimal information about actions and states. In *Proc. KR*.
- [Hu and De Giacomo, 2011] Hu, Y. and De Giacomo, G. (2011). Generalized planning: Synthesizing plans that work for multiple environments. In *Proc. IJCAI*, pages 918–923.
- [Jansen et al., 2025] Jansen, N., Gösgens, J., and Geffner, H. (2025). Learning lifted action models from traces of incomplete actions and states. In *Proc. KR*.
- [Karia and Srivastava, 2021] Karia, R. and Srivastava, S. (2021). Learning generalized relational heuristic networks for model-agnostic planning. In *AAAI*.
- [Lamanna et al., 2021] Lamanna, L., Saetti, A., Serafini, L., Gerevini, A., and Traverso, P. (2021). Online learning of action models for pddl planning. In *Proc. IJCAI*, pages 19–27.
- [Morris et al., 2019] Morris, C., Ritzert, M., Fey, M., Hamilton, W. L., Lenssen, J. E., Rattan, G., and Grohe, M. (2019). Weisfeiler and leman go neural: Higher-order graph neural networks. In *Proc. AAAI*, pages 4602–4609.

- [Nuñez-Molina and Geffner, 2025] Nuñez-Molina, C. and Geffner, H. (2025). From next token prediction to (strips) world models. *arXiv preprint arXiv:2509.13389*.
- [Rodriguez et al., 2021] Rodriguez, I. D., Bonet, B., Romero, J., and Geffner, H. (2021). Learning first-order representations for planning from black-box states: New results. In *KR*. arXiv preprint arXiv:2105.10830.
- [Scarselli et al., 2008] Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M., and Monfardini, G. (2008). The graph neural network model. *IEEE transactions on neural networks*, 20(1):61–80.
- [Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *CoRR*, abs/1707.06347.
- [Srivastava et al., 2008] Srivastava, S., Immerman, N., and Zilberstein, S. (2008). Learning generalized plans using abstract counting. In *Proc. AAAI*, pages 991–997.
- [Srivastava et al., 2011a] Srivastava, S., Immerman, N., and Zilberstein, S. (2011a). A new representation and associated algorithms for generalized planning. *Artificial Intelligence*, 175(2):615–647.
- [Srivastava et al., 2011b] Srivastava, S., Zilberstein, S., Immerman, N., and Geffner, H. (2011b). Qualitative numeric planning. In *AAAI*.
- [Ståhlberg et al., 2021] Ståhlberg, S., Bonet, B., and Geffner, H. (2021). Learning general optimal policies with graph neural networks: Expressive power, transparency, and limits. *arXiv preprint arXiv:2109.10129*.
- [Ståhlberg et al., 2022a] Ståhlberg, S., Bonet, B., and Geffner, H. (2022a). Learning general optimal policies with graph neural networks: Expressive power, transparency, and limits. In *Proc. ICAPS*.
- [Ståhlberg et al., 2022b] Ståhlberg, S., Bonet, B., and Geffner, H. (2022b). Learning generalized policies without supervision using gnns. In *Proc. KR*.
- [Ståhlberg et al., 2023] Ståhlberg, S., Bonet, B., and Geffner, H. (2023). Learning general policies with policy gradient methods. In *Proc. KR*.
- [Ståhlberg et al., 2025] Ståhlberg, S., Bonet, B., and Geffner, H. (2025). Learning more expressive general policies for classical planning. In *Proc. AAAI*.
- [Sutton and Barto, 2018] Sutton, R. S. and Barto, A. G. (2018). *Reinforcement learning: An introduction*. MIT Press.

- [Toenshoff et al., 2021] Toenshoff, J., Ritzert, M., Wolf, H., and Grohe, M. (2021). Graph neural networks for maximum constraint satisfaction. *Frontiers in artificial intelligence*, 3:98.
- [Toyer et al., 2020] Toyer, S., Thiébaux, S., Trevizan, F., and Xie, L. (2020). Asnets: Deep learning for generalised planning. *Journal of Artificial Intelligence Research*, 68:1–68.
- [Toyer et al., 2018] Toyer, S., Trevizan, F., Thiébaux, S., and Xie, L. (2018). Action schema networks: Generalised policies with deep learning. In *AAAI*.
- [Williams and Peng, 1991] Williams, R. J. and Peng, J. (1991). Function optimization using connectionist reinforcement learning algorithms. *Connection Science*, 3(3):241–268.
- [Xu et al., 2019] Xu, K., Hu, W., Leskovec, J., and Jegelka, S. (2019). How powerful are graph neural networks? In *ICLR*.
- [Yang et al., 2024] Yang, A., Chiang, D., and Angluin, D. (2024). Masked hard-attention transformers recognize exactly the star-free languages. In *Proc. NeurIPS*.